

DB(Database)

20/09/03 ~ 20/12/16

박지원

목차

1-1강_Database Overview

1-2강_데이터 추상화 및 데이터 모델

1-3강_데이터베이스 시스템

2-1강_관계형 데이터베이스 모델

2-3강_관계 대수

2-4강_추가 관계 대수

3-1강_데이터베이스 언어

3-2강_DDL

3-3강_DML

3-4강_Null Values

5-1강_집계 함수

5-2강_조인 테이블

5-3강_중첩 질의

6-1강_View

6-2강_Integrity Constraints

6-3강_Trigger

6-4강_Table Authorization

6-5강_View Authorization

8-1강_Embedded SQL

8-3강_Application Architecture

1번 과제 공부

9-1강_SQL Procedural Extensions

9-2강_Functions and Procedures from SQL:1999

과제 2번 공부

10-1강_Entity and Relationship

10-2강_E-R Diagrams

10-3강_E-R Diagram -> Table

10-4강_Design Issues

10-5강_Extended ER Model

11-1강_Good schema and Bad schema

11-2강_Functional Dependency Theory

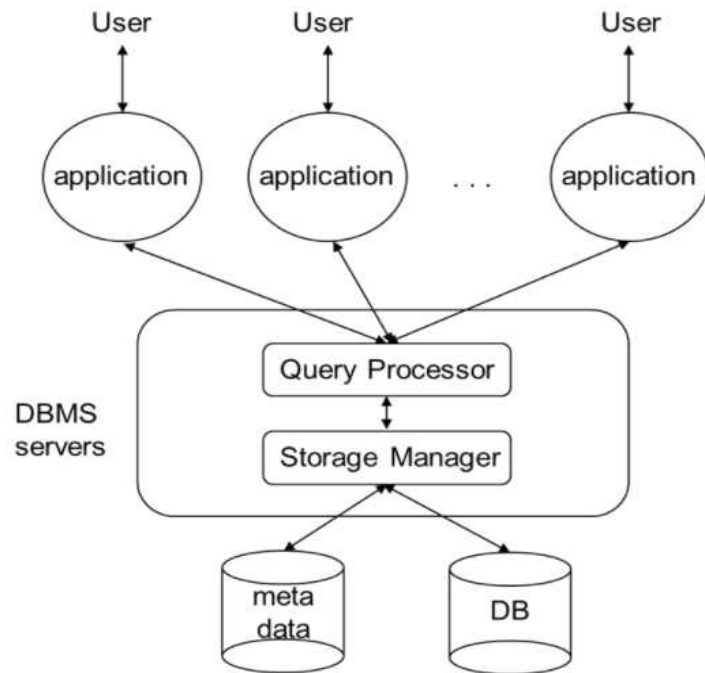
11-3강_Normal Forms

11-4강_Decomposition and Design

<1-1강_Database Overview> 09/03

1. 정의

1) DB(데이터베이스)



- DB의 정의 : 서로 연관 되어 있는 데이터의 모임
 - 메모리에 저장하기에는 대용량이므로 용량이 부족합니다.
 - 디스크에 저장합니다.(HDD , SSD)
 - user의 query를 app을 통해 DBMS로 보내고 DBMS와 DB 사이에 disk i/o가 발생하며 데이터를 가져옵니다.
 - 메모리에 데이터를 저장하는 기술(저용량)은 자료구조 과목입니다.
- <Query Processor>
= 질의와 권한(Authorization)을 담당합니다.
- <Storage Manager>
= 데이터 저장 , 데이터 업데이트 , 데이터 검색(Indexing , Hashing) , 트랜잭션 관리
- DB는 DB로 이루어져 있습니다. 표현이 이상하지만 옳은 표현입니다. 전자 의 DB는 디스크에 존재하는 저장 공간 이며 좌측의 그림에서 의미하는 DB입니다. 후자의 DB는 여러 개의 Table로 이루어진 데이터의 집합입니다. 저장 공간인 DB에는 데이터의 집합인 DB가 여러 개가 저장되어 있습니다.
 - CREATE DATABASE 'DB이름' = DB를 만듭니다. 이 DB를 어떻게 구성할 지가 스키마입니다.

Database defined

A database is an organized collection of structured information, or data, typically stored electronically in a computer system. A database is usually controlled by a **database management system (DBMS)**. Together, the data and the DBMS, along with the applications that are associated with them, are referred to as a database system, often shortened to just database.

- Oracle의 공식자료에서 언급하길 , DBMS와 DB를 크게 구분하지 않고 모두 DB(=database)로 부르기도 합니다.

2) DBMS(database management system)

= 디스크에 존재하는 **DB를 관리하는 소프트웨어**

- 데이터의 종속성과 중복성의 문제를 해결하기 위해 개발되었습니다.
- 응용 프로그램과 데이터사이의 중재자인 매우 크고 복잡한 소프트웨어 입니다.

3) DBS (database system)

= DB + DBMS

4) 객체(Instance)

- = 프로그래밍에서 **변수**에 대응되는 개념입니다.
- = ex) 학생의 키 , 학생의 성적

2. DBMS의 장점(DBMS vs 파일처리)

1) Data abstraction

= 사용자 수준에 따른 추상화가 가능합니다.

2) Real-Time accessing data

= 사용자는 실시간으로 query만 작성하면 데이터에 간단하게 접근 할 수 있습니다.

3) Controlled data redundancy and inconsistency

= 데이터 중복 및 불일치성에 대한 방지가 용이합니다. 특히 불일치성은 매우 중요합니다.

4) Integrity constraint enforcement(Error Checking)

= 대학교 학년은 1,2,3,4만 할당되어야 합니다. 5가 들어오면 ERROR가 발생해야 합니다. 이것을 **무결성(=무결점 , 완전무결)** 이라고 합니다.

= 파일처리 기법으로 파일 관리를 할 때 5학년이 생긴다면 직접 코드를 찾아서 조건문을 변경해야 합니다.

= DBMS를 이용 한다면 직접 코드의 접근이 필요가 없으므로 매우 쉽게 무결성을 보장하기 쉽습니다.(추후 강의)

5) Atomicity of updates

= A계좌에는 100원이 있고 B계좌에는 0원이 있다고 가정합니다. A가 B에게 50원을 준다면 A는 50원 B도 50원이 있어야 합니다. DBMS는 오류가 발생하지 않도록 하기 위해 두 계좌의 합이 언제나 100원이 되는지 보장합니다. 즉, A는 50원 B는 0원이 되는 failure를 막아줍니다.

6) Concurrent access by multiple users

= 동일한 DB에 여러 유저들이 접근할 수 있습니다. A계좌에 100원이 있고 두 명의 유저가 각각 20원 30원씩을 인출했을 때 남은 돈이 언제나 50원임을 보장해줍니다.

7) Data security

= 보안 우수

8) Data backup and recovery

= 백업과 복구 우수

- 파일처리는 위의 특성들을 구현하는 것이 매우 어렵고 성능이 저열합니다.

<1-2강_Data Abstraction & Data Model> 09/03

1. Data Abstraction(데이터 추상화)

= 데이터의 복잡한 내용(details)은 제거하고 가장 중요하고 간단한 내용만으로 데이터를 이해합니다.

- 사용자의 수준에 따라 추상화의 정도를 다르게 할 수 있습니다.

ex) 초등학생한테 컴퓨터를 설명할 때는 본체 마우스 모니터 만 설명하고 , CS전공자한테는 메인 메모리 cpu도 설명을 해주는 차이를 의미합니다.

2. 데이터 추상화 레벨

데이터 추상화 레벨	추상화 정도	단순함 정도	적합 사용자	특징
Physical level	낮음	매우 어렵고 복잡	DB 관리자 , 전문가	하위 단계의 모든 내용 및 physical level 고유 정보 까지 모두 알려줍니다.
Logical level	중간	중간	DB 중수	스키마와 스키마의 정보 수준까지 알려줍니다.
View level	높은	매우 단순하고 간단	DB 처음 배움	복잡한 내부 내용을 감추어 보안에 매우 훌륭 합니다.

3. Data Independence(데이터 독립성)

1) physical data independence

= physical schema의 내용을 변경해도 logical schema와 view schema에 영향을 미치지 않습니다.

2) logical data independence

= logical schema의 내용을 변경해도 view schema에 영향을 미치지 않습니다.

4. 다양한 Database

1) Overview

= 데이터를 표현하는 다양한 방식입니다.

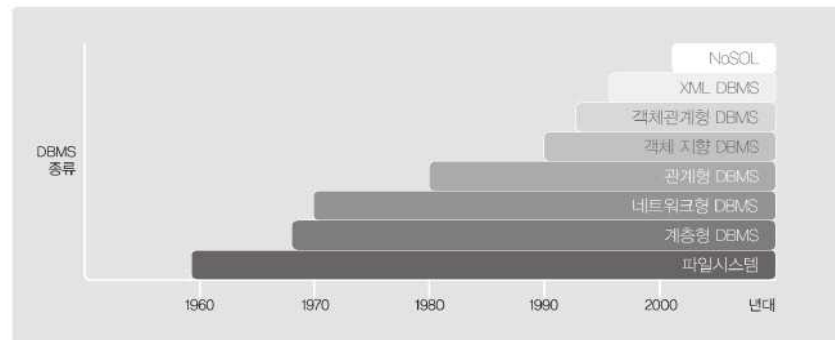
= 실세계를 다루는 프로그램 -> c로 학생 데이터를 표현해야함 -> struct로 구현합니다.

= 관계형 데이터 모델은 relation을 이용

= 데이터(학생), 데이터의 관계(relationships 학생 과 과목), data semantics , 데이터 제약사항(constraints 학생은 1~4학년만 가능),

= 복잡한 데이터를 abstract하여 단순하게 데이터를 나타냅니다.

2) 종류



[그림 4] 데이터베이스의 시대별 응용분야

- ① Relational database management system(=RDBMS) (=관계형 데이터베이스 관리 시스템) : MySQL
- 1970년대에 개발되어 지금까지도 사용되고 있는 근본 DBMS
 - Query Language(질의어)가 존재하여 , 간단한 문법만 익히면 누구나 원하는 DB를 사용할 수 있습니다.
 - 2강 ~ 5강에서 자세하게 배웁니다.

- ② Object-oriented database management system(=OODB) (=객체 지향형 데이터베이스 관리 시스템)
- 1990대에 개발되어 사용하고 있습니다.
 - 클래스 이용하기 때문에 사용할 수 있는 자료형이 매우 다양합니다.
 - 사용자 지정 데이터 타입의 지원 및 상속성의 명세가 가능하여 비정형 복합 정보의 모델링이 가능합니다.

- ③ Object-relation database management system (=ORDB) (=객체 관계 데이터베이스 관리 시스템)
- = 객체 지향 데이터 모델의 장점을 관계형 데이터 모델에 접목 시킨 모델입니다.

- ④ Entity-Relationship(ER) data model
- = DB 디자인
- 10강에서 자세하게 배웁니다.

- ⑤ XML(Extensible Markup Language)
- = 웹 데이터를 표현합니다.
- 데이터의 저장과 전달을 위해 제작된 언어

- ⑥ NoSQL
- = SQL을 사용하지 않는 DBMS
- ex) Firebase

<SQL 설치 TIP>

- MySQL Community Server가 DBMS입니다.
- MySQL Workbench는 GUI기반으로 사용자가 DBMS를 쉽게 사용할 수 있게 만들어 준 Application 입니다.
- 다운로드 : <https://dev.mysql.com/downloads/mysql/>
- 설치 방법 : <https://www.youtube.com/watch?v=kh36nZKCsp4>
- program files 와 program data에 MySQL관련된 파일이 이미 있다면 100% 모두 지우고 설치해야 합니다.
- 반드시 컴퓨터의 이름은 영문이어야 합니다. 아니면 설치 마지막에 무조건 오류가 생깁니다.

<1-3강_Database System> 09/10

1. DB를 제작하는 4가지 단계



<DB 설계>

① 개념적 데이터 모델링

- 현실 세계에서 나타나는 정보 구조를 추상적으로 개념화하는 것
- 엔터티, 엔터티의 식별자, 엔터티 간 관계, 관계의 대응 수 및 차수, 엔터티의 속성 등이 도출됨
- 일반적으로 개체-관계(Entity-Relationship: ER) 모델로 표현됨

② 논리적 데이터 모델링

- 사람의 이해를 위한 개념적 설계의 결과를 데이터베이스 저장에 용이한 논리적 구조로 변환하는 것
- 관계형, 네트워크형, 계층형, 객체 지향형 모델이 있으며, 이 중 관계형 모델이 가장 일반적으로 사용됨
- 관계형 모델의 경우 논리적 설계 단계에서 테이블명, 기본키(Primary Key: PK), 외래키(Foreign Key: FK) 등이 결정됨
- 단계 산출물로 논리 스키마가 도출됨
- 요구 수준에 맞춘 정규화가 수행됨

③ 물리적 데이터 모델링

- 논리적 구조 설계를 통해 생성된 데이터베이스의 물리적 저장 구조 결정
- 열(Column)의 데이터 형식, 제약조건 및 특정 데이터에 접근하기 위한 접근 방법, 접근 경로를 정의함
- 성능 요구사항에 따라 구조를 변환하는 기술이 필요함
- 구체적으로 트랜잭션 분석, 뷰 설계, 인덱스 설계, 용량 설계, 접근방법 설계 등이 수행됨
- 단계 산출물로 물리 스키마가 도출됨

2. 중요 용어

1) Transaction

- = 쪼갤 수 없는 업무처리의 단위
 - = 데이터베이스 시스템의 상태를 변화시키는 프로그램의 작업 단위 입니다.
 - 유저간의 상호작용을 Transaction 데이터가 관리합니다.
- ex) 유저간의 거래 내용

2) Data Dictionary(= Data Directory)

- = Meta Data를 저장하는 공간

3) Meta Data

메타데이터(metadata)는 데이터(data)에 대한 데이터이다. 즉, 부가적 정보를 추가하기 위해 그 데이터 뒤에 함께 따라가는 정보를 말한다. '메타'는 '초월'이나 '한층 높은 논리성이 있는'이란 뜻으로서, 메타데이터는 어떤 데이터가 어떤 구조를 하고 있는지, 또 어디에 있는가에 대한 정보를 일컫는다. 예를 들어, 디지털 카메라에서는 사진을 찍어 기록할 때마다 카메라 자체의 정보와 촬영 당시의 시간, 노출, 플래시 사용 여부, 해상도, 사진 크기 등의 사진 정보를 화상 데이터와 같이 저장하게 되어 있다.

- = 일반적인 데이터(ex) : 학생의 학번)에 대한 데이터
- = "Karen Coyle"의 정의 : 어떤 목적을 가지고 만들어진 데이터
- 일반 데이터는 Database에 저장되고 , 일반적인 데이터에 대한 데이터인 Meta Data는 Database 가 아닌 Data Dictionary에 저장됩니다.

<Meta Data에 저장하는 Data>

- 도메인의 타입
- 두 데이터간의 관계
- Integrity constraints (제약)
- Authorization (권한)
- Statistical Data (통계 데이터) : Table이 몇 개의 tuple을 갖고 있는지.
- Physical file organization information : 데이터 들이 물리적인 파일로 저장되어 있어야 하는데 어떤 형태(b+ tree , hash)로 저장 되어 있는지에 대한 정보.

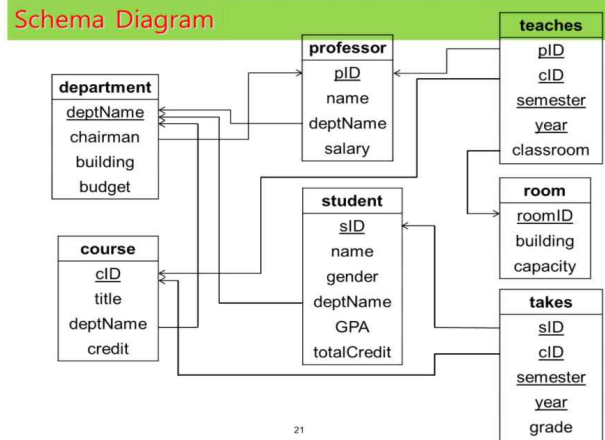
4) 스키마(Schema)

- = 간단한 개념 : 어떤 구조로 데이터가 테이블에 저장되는가를 나타내는 데이터베이스의 논리적 구조
- = 자세한 개념 : 테이블은 사용자가 입력한 데이터를 저장하는 행과 열로 이루어진 저장장소 일 뿐이며 , 이 테이블이 어떤 속성으로 구성되어 있고 , 속성들이 어떤 타입 이며 , 속성들 간에 어떤 관계를 갖고 있고 , 어떤 속성이 primary key인지에 대한 정보를 저장하는 것은 스키마입니다.
- 메타 데이터로 이루어져 있습니다.
- 3 개의 스키마(view , logical , physical)로 구분할 수 있습니다.
- 스키마의 집합을 Catalog라고 합니다.

<Example_1 : 쇼핑몰을 만들어봅시다.>

- = 만약 제가 쇼핑몰 사장이라면 쇼핑몰을 운영하기 위해 관련 데이터를 저장하기 때문에 회원 Table , 상품 Table , 주문내역 Table이 필요 하게 됩니다. 회원 Table에는 이름 , 주소 , 전화번호의 속성을 가져야 하며 ‘이름은 최대 10글자의 가변길이 문자열’, ‘주소는 몇 글자 이상’, ‘전화번호에는 숫자만 기재가능’ 같은 제약 조건을 적습니다. 상품 Table . 주문내역 Table도 마찬가지로 작성해줍니다. 상품 Table은 회원 Table과 어떤 관계가 있는지도 작성합니다. 이처럼 DB에 저장할 Table들을 어떻게 구현할 것인지를 고려하는 것이 스키마 입니다.
- = 정리하면 , **쇼핑몰 사장이 쇼핑몰 DB를 어떻게 구현할지에 대한 모든 내용이 스키마**입니다.

<Example_2 : 학생 스키마>



<Meta data vs Schema>

Metadata is 'data about data'. Whereas **Schema** is the structure/layout of the system.

Real world example for Metadata: The extra information generated when you take a picture with your phone such as date, location, etc.

Real world example for Schema: The layout of a website such as where is the main title, content, etc.

In the context of a database system, **schema is a subset of metadata**.

A database metadata consists of lots of information about its schema, storage, language, security, etc. Whereas a database schema consists of the structure of data in the database such as how the data is organized into tables, rows and columns, etc as well as the relationships and constraints between those data.

- Schema는 DB의 구조지만 , Meta Data는 결국 데이터입니다.

<2-1강_Relational Database> 09/17

1. RDBMS (관계형 데이터베이스)

= Relations(table) + Integrity constraint(다수의 테이블과 제약 조건으로 구성되어 있습니다.)

- 컴퓨터수학에서 배우는 Relation 개념과 매우 밀접하게 연관되어 있습니다.
- 테이블과 다른 테이블의 관계를 표시하며 DB를 구성합니다. 관계를 나타낼 때 외래기를 사용합니다.
- 2차원 배열(Table의 구조)에 정보를 저장하는 형식입니다.
- 멀티미디어 같은 비정형 복합 정보의 처리가 어렵습니다.

<관계(=Relation)>

- 가장 쉽게 이해하는 방법은 관계는 그래프에서 Edge 입니다.
- 여담으로 , 어떤 집합의 원소는 그래프에서 Node(=Vertex) 입니다.
- 집합 A는 컴퓨터학부 학생들의 집합이라 하고 집합 B는 컴퓨터학부 전공과목의 집합이라고 합시다. a는 컴퓨터 학부 학생이고 b는 전공과목 이라고 합시다. 'a는 과목 b를 수강 한다'로 구성되어 있는 관계를 R이라고 합시다. (박지원 , 컴퓨터수학2)는 R에 포함 되어 있습니다. 만약 a는 컴퓨터 학부 학생이며 여자인 학생이고 b는 전공과목인 관계를 R2라 하면 (박지원 , 컴퓨터수학2)는 R2에 포함 되지 않습니다.
- 집합 A에서 집합 B로의 관계는 집합 $A \times B$ 의 부분집합입니다.

2. 용어

1) table = relation

<학생>

학번	이름	학년	학과
20357	홍길동	1	영문학
20264	성준향	3	경영학
20154	이몽룡	4	전산학

Relation or Table: 학생

2) tuple = row = record

<학생>

학번	이름	학년	학과
20357	홍길동	1	영문학
20264	성준향	3	경영학
20154	이몽룡	4	전산학

Cardinality: 3

Tuple

- Cardinality는 Tuple의 개수입니다.

- tuple의 중복은 허용하지 않습니다.

3) attribute = column = 속성

학번	이름	학년	학과
20357	홍길동	1	영문학
20264	성춘향	3	경영학
20154	이몽룡	4	전산학

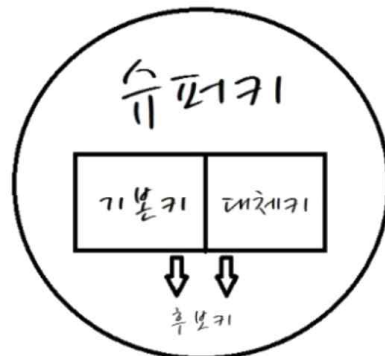
- Degree는 Attribute의 개수입니다.

Attribute: 학번, 이름, 학년, 학과

Degree: 4

- **Attribute**는 **조깔 수 없는 값이어야 합니다.** ex) int , time , date , real(실수)
- Set , Multi-Set, List 같은 container는 조깔 수 있으므로 attribute가 될 수 없습니다.
- 여러 가지 Key가 존재 합니다.
- 속성 이름은 중복을 허용하지 않습니다.
- **Null 값(속성 값이 비어있음)**을 가질 수 있습니다.

4) Key



- Reference : <https://limkydev.tistory.com/108>

<유일성>

= 주민등록번호처럼 DB에서 해당 속성의 값이 절대 겹치지 않습니다.

<최소성>

= 예를 들어 학번(유일성 만족) + 주민등록번호(유일성 만족)로 Key를 만들면 유일성은 만족하지만 학번, 주민등록번호만 독립적으로 Key로 사용해도 유일성을 만족합니다. 그렇기 때문에 학번 + 주민등록번호처럼 낭비하는 경우는 최소성을 만족하지 않습니다.

- Super Key를 이해하는데 위의 설명이 큰 도움이 됩니다.

① Candidate Key(후보키)

= Primary Key가 될 수 있는 조건을 만족한 Key.

- 여러 개일 수 있으며 테이블에 반드시 1개는 있어야 합니다.

- 유일성과 최소성을 모두 만족시킵니다.

② Primary Key(기본키)

= Candidate Key 중에서 DB 설계자가 정한 가장 대표적인 Key.

- 테이블에 반드시 1개만 존재해야 합니다.

- Null 값을 가질 수 없습니다.

- Primary Key로 지정된 속성에는 동일한 값이 중복되어 저장될 수 없습니다. (ex : 주민번호는 중복이 불가능)

- 유일성과 최소성을 모두 만족시킵니다.

- 여러 속성으로 구성 될 수 있습니다. (ex : '이름 + 학번'이 Primary Key)

- 값이 변경되면 primary key가 유일한 값인지를 검사해야 하기 때문에 변경이 적은 속성이 유리합니다.

③ Super Key(슈퍼키)

= 하나 이상의 속성들을 조합하여 유일성을 만족시키는 Key.

- 유일성은 만족시키지만 최소성은 만족시키지 않습니다.

- Relation DB Model 내부에서 보통 매우 많이 존재합니다.

④ Compositive Key(합성키)

= 두 개 이상의 속성으로 이루어진 Key.

= 학생 테이블에서 Major와 학점 각각은 Super Key가 될 수 없습니다. 하지만 둘의 데카르트 곱을 하여 둘을 합친다면 Super Key가 될 수 있습니다.

⑤ Alternative Key(대체키 OR 보조키)

= Candidate Key 중에서 Primary Key가 아닌 모든 Key.

- 유일성과 최소성을 모두 만족시킵니다.

⑥ Foreign Key(외래키)

= 서로 다른 두개의 테이블을 연결해주는 연결 다리 역할을 하는 Primary Key

- 하나의 table만 사용해서 데이터를 분석할 때는 큰 의미가 없지만 , 두 개 이상의 table을 조합해서 데이터를 분석할 때에는 중요한 개념입니다.

- SQL에서 가장 주의해야 할 일은 tuple의 중복입니다. 다양한 속성을 가진 tuple에서 1개의 속성만이 바뀐 tuple을 계속 추가하다 보면 다른 속성들은 모두 중복이 되어서 Table의 크기가 엄청 커지게 되고 , 공간의 낭비가 생깁니다. 이를 방지하게 위해 table을 쪼개서 두 개의 table로 저장
을 하고 필요할 때 마다 참조를 합니다.

- 참조를 할 때 참조의 매개 Key가 다양하면 안 됩니다. 그러므로 참조의 매개는 반드시 Primary Key로 합니다.

<Example_1>

teaches		
pID	cID	...
100	CS101	...
100	CS201	...
200	EE101	...

professor		
pID	name	...
100	Kim	...
200	Lee	...

```
Create table teaches (
  pID      varchar(5),
  cID      varchar(5),
  ...
  primary key(pID, cID, semester, year)
  foreign key(pID) references professor,
  foreign key(cID) references course);
```

- teaches의 속성 pID는 반드시 professor 테이블에 존재해야 하므로 pID는 foreign key입니다.
- teaches의 속성 cID는 반드시 course 테이블에 존재해야 하므로 cID는 foreign key입니다.

<Example_2 : 쇼핑물>

- Reference : <https://brunch.co.kr/@dan-kim/26>

user_id	user_pwd	name	gender	address	order_date	product_id	product_title	qty
marco117	xxx	Marco	male	451 Sierra St.	2019-10-01	p131	coke	5
marco117	xxx	Marco	male	451 Sierra St.	2019-10-01	p132	sprite	10
marco117	xxx	Marco	male	451 Sierra St.	2019-10-02	p131	coke	1
marco117	xxx	Marco	male	451 Sierra St.	2019-10-02	p132	sprite	5

= Marco117의 아이디를 가진 사람이 물건을 계속 주문하면 붉은색으로 표기된 속성들은 중복되어서 table을 채우는 낭비를 발생시킵니다.

order_d...	product_id	product_title	qty	user_id
2019-10-01	p131	coke	5	marco171
2019-10-01	p132	sprite	10	marco171
2019-10-02	p131	coke	1	marco171
2019-10-02	p132	sprite	5	marco171

user_id	user_pwd	name	gender	address
marco117	xxx	Marco	male	451 Sierra St.

= 2개의 테이블로 구분하고 , Primary Key인 user_id를 Foreign Key로 설정하여 참조하도록 합니다.

5) Domain(정의역)

= Attribute가 가질 수 있는 모든 값들의 집합

- 모든 domain에 null값을 가집니다. 다시 말해 , 모든 domain에는 숨겨진 null 원소가 존재합니다.

ex) 성별 : 남 , 여 , null

6) Instance

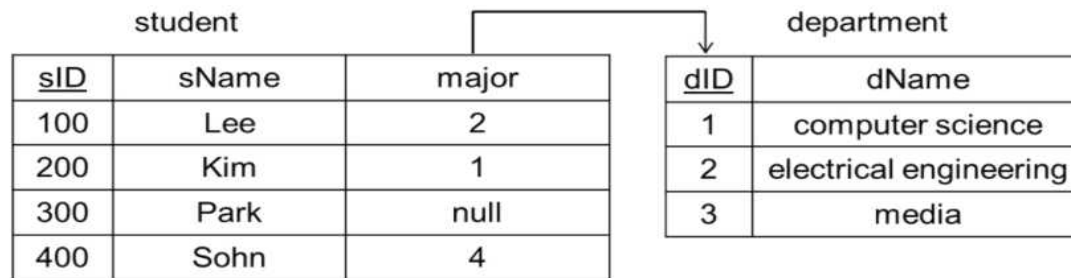
= Table은 계속 값이 변합니다. 어떤 시점의 Table의 상태를 instance 라고 합니다.

- Domain의 부분 집합 입니다.

3. Referential Integrity Constraint

- = 참조 무결성 제약 조건
- = **두 테이블 간에** 존재하는 제약 조건
- **값을 이용**해서 표현합니다.
- 6-2강에서 자세하게 배웁니다.

<Example>



- major는 department table을 반드시 참조해야 합니다. 그러므로 major는 1,2,3의 값이 와야 합니다.
- 모든 domain은 null 값을 갖기 때문에 major에 null을 할당 한 것은 옳은 방법 입니다. 예시에서 Park은 아직 학과를 받지 못한 학생이라고 생각하면 됩니다.
- Sohn은 major가 4이므로 **제약조건을 위배하므로 Student table은 잘못 만들어 졌습니다.**
- Referencing relation = 참조하는 테이블 -> student
- Referenced relation = 참조 당하는 테이블 -> department

*** 2-2강은 예시 자료이므로 스킵 합니다. ***

<2-3강_Relational Algebra> 09/25

*** table , tuple , attribute로 통일해서 필기하겠습니다. ***

1. 정의

= Relational Algebra

= 사용자 단계에서 사용자가 query를 작성할 때는 SQL을 사용합니다. DBMS에 SQL로 만들어진 query가 전달되면 DBMS에서 사용되는 언어인 relational algebra로 사용자의 query를 시스템이 해석합니다.

- Procedure language 이므로 연산자의 순서대로 처리 합니다.

- SQL은 non procedure language 입니다. 왜냐하면 사용자가 연산자의 순서까지 지키면서 query를 작성하는 것은 매우 어렵기 때문입니다.

2. 관계 대수 6대 연산자

- select: σ

- project: Π

- union: $\cup$

- set difference: $-$

- Cartesian product: $\times$

- rename: ρ

- 6개의 연산자 모두 Input과 Output이 존재하는데 Input과 Output 모두 반드시 Table 입니다.

- 6개의 연산자로 모든 관계 대수식을 표현할 수 있습니다.

- 관계 대수에서 다루는 Table은 모두 Set 기반이므로 중복을 허용하지 않습니다. 언제나 중복을 제거합니다. (예외 : Multi-Set기반 Table)

- 비교 연산자 : $AND(\wedge)$, $OR(\vee)$, $NOT(\neg)$

1) select

Relation r

A	B	C	D
α	α	1	5
α	α	5	2
β	β	12	12
α	β	23	10

$\sigma_{A=B \wedge D > 4}(r)$

A	B	C	D
α	α	1	5
β	β	12	12

= 선택 연산

= 연산기호는 시그마 입니다.

- 4개의 tuple 4개의 속성으로 이루어진 테이블에서 $A=B$ and $D>4$ 인 tuple을 선택하는 그림입니다.
- 1행과 3행의 tuple을 골라서 새로운 테이블을 완성시켰습니다.
- Select에서는 비교 연산자와 함수를 사용할 수 있습니다.

2) project

Relation r

A	B	C	D
α	α	1	5
α	α	5	2
β	β	12	12
α	β	23	10

$\Pi_{A,C}(r)$

A	C
α	1
α	5
β	12
α	23

$\Pi_{A,B}(r)$

A	B
α	α
β	β
α	β

= 투사 연산

= 연산기호는 파이 입니다.

- 파이A,C(r)은 테이블 r에서 **A속성과 C속성만 선택해서** 새로운 테이블을 만드는 연산입니다.

- 파이A,B(r)에서 (a,a)는 duplicated rows(중복되는 tuple)이므로 제외합니다.

3) Union

Relation r

A	B
α	1
α	5
α	23

Relation s

A	B
α	1
β	12

$r \cup s$

A	B
α	1
α	5
α	23
β	12

- = 합집합 연산
- = 두 테이블을 합쳐서 새로운 테이블을 만듭니다.
- 연산기호는 U입니다.
- 반드시 **Same Arity**(속성의 개수가 같아야 합니다.)
- 반드시 **Type Compatible**(속성의 도메인이 같아야합니다.) -> r의 도메인은 A,B이고 s의 도메인은 A,B이므로 서로 같습니다.
- 유니온 된 테이블도 기본적으로 Set이므로 **중복을 제거** 합니다.

4) set difference

Relation r

A	B
α	1
α	5
α	23

Relation s

A	B
α	1
β	12

$r - s$

A	B
α	5
α	23

$s - r$

A	B
β	12

- = 차집합 연산
- = 연산의 앞에 있는 테이블에서 연산의 뒤에 있는 테이블과 겹치는 tuple을 제거 합니다.
- 연산기호는 - 입니다.
- 반드시 **Same Arity , Type Compatible**
- 차집합 된 테이블도 기본적으로 Set이므로 **중복을 제거** 합니다.

5) Cartesian product

Relation r

A	B
α	1
α	5
α	23

Relation s

C	D
α	1
β	12

r x s

A	B	C	D
α	1	α	1
α	1	β	12
α	5	α	1
α	5	β	12
α	23	α	1
α	23	β	12

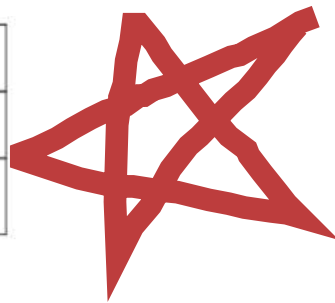
= 두 테이블의 모든 tuple을 곱하여 새로운 tuple을 만들어주고 그 들로 이루어진 테이블을 결과로 도출합니다.

- 연산기호는 X 입니다.

- 만약 relation p의 도메인이 (A,B)이고 relation q의 도메인이 (A,C)일 때 두 테이블을 카티시안 곱 연산을 한다면 A 속성이 겹치므로 반드시 rename 연산을 통해 A1 , A2로 변경해 주어야 합니다.

<특성>

	R	S	R x S
Arity	m	n	m+n
# of tuples	x	y	x*y



- 새로운 테이블의 속성의 개수 = 두 테이블의 속성의 개수의 합

- 새로운 테이블의 tuple의 개수 = 두 테이블의 tuple의 개수의 곱

- Cartesian 연산을 수행해서 나온 테이블은 쓸데없는 tuple까지 포함하기 때문에 크기가 매우 커집니다.

데카르트 곱 = 카테시안 곱

방금 언급했다시피 학교에서 수학을 배웠던 사람들 중 나를 포함한 상당수는 그냥 데카르트 곱=카테시안 곱이라고만 알고 있는 경우가 많을 것이다. 학교 졸업한지 한참 되었지만 최근에 다시 공부를 하다 보니 문득 궁금해서 찾아봤다.

데카르트는 “나는 생각한다 고로 나는 존재한다”로 유명한 중세 프랑스의 물리학자이자 철학자, 그리고 해석기하학의 창시자로 불리며 풀테임은 르네 데카르트, René Descartes. 데카르트라고 알고 있는 이름의 철자는 Descartes, Des + Cartes.

뜻하는 바는 “of Cartes” (des- 라는 접두어가 of의 의미). Cartes 마을/동네에서 온 사람들이란 뜻. 이게 영어쪽으로 오면서 -ian 을 붙이는(캐네디언 혹은 페르시언 처럼) 식으로 되어서 Cartesian 이 된 듯.

6) rename

- = 재명명 연산
- 연산기호는 $p(\rho)$ 입니다.
- = 테이블이나 속성의 이름을 바꾸고 싶을 때 사용합니다.

3. Relational Algebra Expressions(관계 대수 식)

<Example 1 : “CS”학부에 소속된 남자학생의 이름과 학점을 찾으시오>

= select & projection

- $\Pi_{\text{name, GPA}}(\sigma_{\text{deptName}=\text{"CS"} \wedge \text{gender}=\text{"M"}}(\text{student}))$
- $\Pi_{\text{name, GPA}}(\sigma_{\text{gender}=\text{"M"} \wedge \text{deptName}=\text{"CS"}}(\text{student}))$

- 그림의 두 관계대수식이 모두 가능합니다.
- 주어진 테이블에서 cs , m 을 만족하는 tuple만 뽑아서 테이블을 만들고 거기서 이름과 학점을 project합니다.

<Example 2 : “CS” 학과 OR “EE” 학과에서 개설되는 title을 고르시오.>

= select & union

- $\Pi_{\text{title}}(\sigma_{\text{deptName}=\text{"CS"}}(\text{course})) \cup \Pi_{\text{title}}(\sigma_{\text{deptName}=\text{"EE"}}(\text{course}))$
- $\Pi_{\text{title}}(\sigma_{\text{deptName}=\text{"CS"} \vee \text{deptName}=\text{"EE"}}(\text{course}))$

- OR 연산이므로 Union을 이용합니다.

<Example 3 : “CS” 학과 AND “EE” 학과에도 개설되는 title을 고르시오>

- $\Pi_{\text{title}}(\sigma_{\text{deptName}=\text{"CS"} \wedge \text{deptName}=\text{"EE"}}(\text{course}))$ (wrong one!!!)

- 교집합 연산을 쓰면 좋을 것 같은데 6개의 연산에 존재하지 않습니다. 확장된 연산자에는 교집합이 존재하고 나중에 배웁니다.
- 위의 그림은 잘못된 식이고 저 연산은 교집합을 의미하지 않습니다. 결과로는 공집합이 나옵니다.

<Example 4 : 두 개의 학과가 있을 때 1번 학과의 예산이 2번 학과의 예산 보다 큰 경우를 구하시오>
= 이 문제는 무조건 Cartesian product를 이용해서 구하면 됩니다.

<2-4강_Additional Relational Algebra> 09/26

- 기본적인 6개의 연산자로 모든 관계대수식을 표현할 수 있지만 , 더 편리하게 나타내기 위해서 5개의 추가 연산자를 배워봅시다.
- 5개의 추가적인 관계 대수 연산자는 없어도 지장은 없습니다.

1. Assignment= 할당 연산

- = 연산 기호는 $\leftarrow$ 입니다.
- = 변수의 초기화처럼 사용합니다.

2. Set Intersection

- = 교집합 이며 , 겹치는 tuple로 테이블을 만듭니다.
- = 연산 기호는 $\cap$ 입니다.
- 반드시 Same Arity , Type Compatible

3. Natural Join

- = 입력된 테이블에서 , 겹치는 속성들의 값이 같은 tuple 만을 선택해서 합치고 , 이 tuple들로 새로운 테이블을 만듭니다.

- = 연산 기호는  입니다.

- 5-2강에서 배우지만 Natural Join은 Inner join(join type) + Natural(join condition)이 합쳐진 연산입니다.
- Cartesian은 모든 tuple들의 조합으로 새로운 테이블을 만든다면 , Natural Join은 겹치는 속성들의 값이 같은 tuple들만의 조합으로 새로운 테이블을 만듭니다.

<Example 1>

Relation r

A	B	C	D
α	1	α	a
β	2	γ	a
γ	4	β	b
α	1	γ	a
δ	2	β	b

Relation s

B	D	E
1	a	α
3	a	β
1	a	γ
2	b	δ
3	b	δ

$r \bowtie s$

A	B	C	D	E
α	1	α	a	α
α	1	α	a	γ
α	1	γ	a	α
α	1	γ	a	γ
δ	2	β	b	δ

= 두 테이블에서 속성 B와 D가 겹칩니다.

- B와 D 속성의 값이 같은 tuple은 (1,a) 또는 (2,b)를 갖고 있습니다. 그러므로 (1,a)의 tuple은 4개 이고 (2,b)의 tuple은 1개 이므로 총 5개의 tuple을 갖는 테이블이 결과가 테이블이 됩니다.

<Example 2 : 2020년 가을에 강의를 한 교수 이름 + 그 교수가 강의한 교과목을 알고 싶습니다.>

= teaches 테이블과 professor 테이블과 course 테이블을 join하면 됩니다.

- professor 테이블에 department_name 속성이 있고 course 테이블에도 department_name 속성이 있으므로 course 테이블을 my_course 테이블로 rename 해주어야 합니다. 다시 말해, 속성의 이름이 겹치면 Default로 자연 조인하기 때문에 자연 조인시키고 싶지 않으면 겹치는 속성의 이름을 Rename 해주어야 합니다.

4. Outer Join

= natural join 뿐만 아니라 공통되지 않은 tuple도 합친 테이블 입니다.

- 연산기호는  입니다.

- Join 결과 존재하지 않는 속성은 null로 표기 합니다.

<myProfessor 테이블과 myTeaches 테이블>

myProfessor

pID	name	deptName
10101	Sam	Computer
12121	Wu	Finance
15151	Mozart	Music

myTeaches

pID	courseID
10101	CS-101
12121	FIN-201
86868	BIO-101

1) Natural Left Outer Join

myProfessor ⋈ myTeaches

plD	name	deptName	courseID
10101	Sam	Computer	CS-101
12121	Wu	Finance	FIN-201

myProfessor ⋈_L myTeaches

plD	name	deptName	courseID
10101	Sam	Computer	CS-101
12121	Wu	Finance	FIN-201
15151	Mozart	Music	null

- 1번 표는 Natural Join의 결과입니다.

- 2번 표는 Natural Left Outer Join의 결과 입니다. myProfessor에서 myTeaches와 match했을 때 값이 같은 tuple의 합집합과 값이 다른 tuple의 합집합을 모두 포함하는 테이블입니다.

2) Natural Right Outer Join

myProfessor $\bowtie$ myTeaches

pID	name	deptName	courseID
10101	Sam	Computer	CS-101
12121	Wu	Finance	FIN-201
86868	null	null	BIO-101

myProfessor $\Join$ myTeaches

pID	name	deptName	courseID
10101	Sam	Computer	CS-101
12121	Wu	Finance	FIN-201
15151	Mozart	Music	null
86868	null	null	BIO-101

- 1번 표는 Natural Right Outer Join의 결과 입니다. myTeaches에서 myProfessor과 match했을 때 값이 같은 tuple의 합집합과 값이 다른 tuple의 합집합을 모두 포함하는 테이블입니다.
- 2번 표는 Natural left Outer Join 과 Natural Right Outer Join을 합친 Outer Join 입니다.

5. Division

= 입력으로 두 개의 테이블이 주어지고 , 한 테이블은 나누어지는 분자 역할을 하고 다른 한 테이블은 나누는 분모 역할을 합니다. 다시 말해 , **큰 테이블에서 작은 테이블을 나누는 연산**입니다.

- 연산기호는 $\div$ 입니다.
- 분자 테이블을 R , 분모 테이블을 S라 하면 , R의 속성에는 S의 속성이 모두 포함되어야 합니다.
- R의 도메인이 3개이고 , S의 도메인이 1개라면 결과 테이블의 도메인은 3-1로 2개 입니다. (Set Difference)
- R 에서 S를 나누는 연산이므로 결과 테이블에는 S의 도메인이 포함되지 않습니다.
- 실전에서 정말 많이 쓰이는 중요한 연산입니다.

<Example 1>

A	B	B	B	A	A
α	1	1	1	α	α
α	2	2	2	β	
α	3		3		
β	2				
β	1				
δ	1				
δ	3				
δ	4				
γ	6				
γ	1				

r

B
1
2

s1

B
1
2
3

s2

$r \div s1$ $r \div s2$

- = $r \div s1$ = 속성 B의 1 과 2를 모두 갖는 A의 원소 = 알파와 베타
- = $r \div s2$ = 속성 B의 1 과 2 와 3을 모두 갖는 A의 원소 = 알파

<Example 2>

A	B	C	D	E
α	1	α	a	1
α	1	γ	a	1
α	1	γ	b	1
β	1	γ	a	1
β	1	γ	b	3
γ	1	γ	a	1
γ	1	γ	b	1
γ	1	β	a	1

D	E
a	1
b	1

A	B	C
α	1	γ
γ	1	γ

S

$r \div S$

r

$r \div s = (D,E)$ 가 (a,1) , (b,1)을 모두 갖는 A , B , C를 찾습니다. = (알파 , 1 , 감마) , (감마 , 1 , 감마)

<3-1강_DB Languages> 09/30

1. 기능적으로 구분한 DB Languages

= DB Language는 사용자가 DBMS에게 query(질의)를 통해 명령을 내리기 위해 사용하는 언어입니다.

1) DDL(데이터 정의어)

```
Create table professor (  
    pID          char(5),  
    name         varchar(20),  
    deptName     varchar(20),  
    salary       numeric(8,2));
```

= Data Definition Language

= DB Schema를 관리합니다.

= 테이블을 생성하고 조작할 때 사용합니다.

- Domain , Integrity constraint(제약)등을 표현할 수 있습니다.

- DDL compiler 컴파일 합니다.

ex) CREATE , ALTER , DROP

2) DML(데이터 조작어)

= Data Manipulation Language

= DB Instance를 관리합니다.

= DB안에 있는 데이터를 조회하거나 검색할 때 사용합니다.

- Query language(질의어) 라고도 부릅니다.

ex) INSERT , SELECT , UPDATE , DELETE

3) DCL(데이터 제어어)

= Data Control Language

= DB의 **스키마** , **인스턴스를 제외한 다른 객체**를 관리합니다.

= **데이터베이스에 접근하고 객체들을 사용하도록 권한을 주고 회수할 때 사용합니다.**

- 유저가 여러 개의 질의를 요구할 때 DBMS에 session을 만들고 , session 내부에 transaction 들을 저장합니다. 그 transaction들 내부에는 query가 저장되어 있습니다.

- Authorization(권한)을 부여하고 , User Account(유저 정보)도 갖고 있습니다.

ex) GRANT , REVOKE

2. SQL

= Structured Query Language

- Relational database language(관계형 데이터베이스 언어)의 표준입니다.

- IBM에서 최초로 만들었습니다.

- 대소문자를 구분하지 않습니다.

- 세미콜론으로 문장의 끝을 표현합니다.

- **DDL + DML + DCL을 모두 포함**하고 있습니다.

- **Non-Procedural Languages**(비절차적 언어)입니다.

- SQL은 Default로 중복을 허용합니다. 관계 대수는 Default로 중복을 허용하지 않습니다. (헷갈리기 쉬운 개념)

<Non-Procedural Languages>

= C 같은 Procedural Languages는 Query를 작성할 때 어떤 데이터를 찾을 것 인지(What) , 그 데이터를 찾을 때 어떻게 찾을 것 인지(How)를 모두 적어주어야 합니다. 하지만 SQL 같은 Non-Procedural Languages는 **Query를 작성할 때 어떤 데이터를 찾을 것 인지(What)만을 적어주면** 됩니다. 그로 인해 User는 Query를 작성할 때 매우 간단하지만 DBMS는 스스로 How를 만들어야 하므로 DBMS 입장에서는 더 복잡한 언어입니다.

DBMS는 How를 관계 대수로 구현합니다.

= Declarative Language라고도 합니다.

<3-2강_DDL> 09/30

1. CREATE

= 테이블을 만들고 테이블의 스키마를 생성합니다.

Create table R

(A₁ D₁,

...,

A_n D_n,

(integrity-constraint₁),

...,

(integrity-constraint_k));

기호	역할
R	테이블 이름
A _i	속성의 이름
D _i	속성의 타입
integrity-constraint	무결성 제약 -> 6-2강에서 모두 배웁니다.

<Example 1>

```
Create table professor (  
    plD          char(5),  
    name         varchar(20) not null,  
    deptName     varchar(20),  
    salary       numeric(8,2));
```

- Table = professor
- 속성 = plD , name , deptName(학과), salary
- 속성의 타입 = char(5), varchar(20), numeric(8,2)
- Integrity Constraints = not null(이름은 null이면 안 됩니다.)
- Create문의 내용은 속성과 속성의 타입 그리고 제약조건에 해당하는 Meta data이므로 Data Dictionary에 저장됩니다.

<Example 2>

```
Create table professor (  
    pID          char(5),  
    name         varchar(20) not null,  
    deptName     varchar(20),  
    salary       numeric(8,2),  
    primary key (pID),  
    foreign key (deptName) references department);
```

- = 붉은 동그라미는 Integrity Constraints의 예시입니다.
- name은 null이 되면 안 됩니다.
- pID가 primary key입니다.
- deptName은 반드시 department table에 존재해야 합니다.

2. ALTER

- = 테이블에 새로운 속성을 추가하거나 기존 속성을 제거 합니다.
- = 테이블에 새로운 제약조건을 추가하거나 기존 제약조건을 제거 합니다.

- 옵션이 존재합니다.

① add

- = 속성D를 table r에서 추가시킬 때

② drop

- = 속성D를 table r에서 제거시킬 때

3. DROP

= 테이블 자체를 삭제시키므로 테이블의 모든 정보와 스키마 까지 완전 삭제

ex) Drop table student

<DROP VS DELETE>

- DROP은 DDL 명령어 이고 DELETE는 DML 명령어입니다.

- DDL은 테이블 자체를 삭제시키므로 테이블의 모든 정보와 스키마 까지 완전 삭제시키지만 DELETE는 테이블의 모든 tuple(내용물)만 지우는 것이므로 테이블이 empty 상태가 되지만 스키마는 삭제되지 않습니다.

<3-3강_DML> 10/01 ~ 10/08

*** 모든 예시는 교재의 professor table을 사용했습니다. ***

1. SELECT

```
select A1, A2, ..., An      // attribute list
from R1, R2, ..., Rm       // relation list
where P                     // selection predicate
group by <grouping attributes>
having <conditions>
order by <ordering attributes>;
```

1) Select

- = 테이블에서 **tuple의 특정 속성을 선택해서 테이블을 출력할 때** 사용합니다. 그 속성들로 이루어진 테이블이 결과로 출력됩니다.
- 관계 대수의 Projection과 유사합니다.

<Example>

① Select name from professor

= professor table에서 tuple들의 name 속성만 선택해서 테이블을 출력합니다.

② Select *

= tuple의 모든 속성을 선택해서 테이블을 출력합니다.

③ Select salary/12 from professor

= professor table에서 tuple의 월급(속성)의 값을 12로 나눈 값을 선택해서 테이블을 출력합니다.

43	
44	SELECT ENAME, JOB FROM EMP;
45	
46	
47	
48	

질의 결과 x

SQL | 인출된 모든 행: 14(0초)

	ENAME	JOB
1	SMITH	CLERK
2	ALLEN	SALESMAN
3	WARD	SALESMAN
4	JONES	MANAGER
5	MARTIN	SALESMAN
6	BLAKE	MANAGER
7	CLARK	MANAGER

<Duplicate(중복)>

- SQL은 Default로 중복되는 tuple을 허용합니다. 그로 인해 사용자는 SELECT 할 때 중복되는 tuple을 허용하거나 허용하지 않을 수 있습니다.
- Select distinct deptName from professor = distinct를 사용하면 tuple의 중복을 허용하지 않습니다.
- Select all deptName from professor = all을 사용하면 tuple의 중복을 허용합니다.
- all 이 Default입니다.

2) From

- = 선택을 할 때 어떤 테이블에서 선택하는지 나타냅니다.
- 한 개 또는 여러 개의 테이블에서 선택 할 수 있습니다.

<Example>

```
Select *  
from professor, teaches;
```

- = professor와 teaches 테이블을 **Cartesian product** 한 테이블에서 선택합니다.
- Cartesian product의 결과는 너무 크기 때문에 의미 없는 데이터가 많습니다. 그러므로 **where와 같이 사용하여 의미 있는 tuple만을 선택합니다.**

3) Where

- = 특정 **조건을 만족시키는 속성**을 선택할 때 사용합니다. 다시 말해, 참과 거짓을 결정하는 **조건문**입니다.
- 조건이 참 일 때 선택하고, 조건이 거짓이면 선택하지 않는 방식이며, 이는 일반적인 SQL의 Execution Model(실행 모델)입니다.

<Between>

```
where salary between 5000 and 6000;
```

= 5000 <= Salary AND Salary <= 6000

<기초 구문 3가지를 모두 포함하는 SELECT>

```
- Select name  
from professor  
where deptName = 'CS' and salary > 8000;
```

= professor table에서 deptName(속성)이 'CS'이고 salary(속성)가 8000보다 큰 tuple에서 이름을 선택해서 테이블을 만듭니다.

4) Joins

= 두 개의 테이블을 합칠 때 사용.

- 반드시 where 와 같이 사용해서 필요한 속성만 뽑음

① Cartesian product

= 두 테이블의 속성을 모두 합쳐서 결과 테이블을 만듭니다.

- `select * from table_r , table_s where ...` 은 Cartesian Product 입니다.

- 공통 속성을 두 번 모두 적어줍니다.

② Natural Joins

= 두 테이블의 속성을 모두 합쳐서 결과 테이블을 만듭니다.

- 공통 속성을 한 번만 적어줍니다.

5) Rename

= 테이블이나 속성의 이름을 바꿉니다.

= as를 이용합니다.

- `salary/12 as month_salary` -> 속성 salary/12의 이름을 속성 month_salary로 바꿉니다.

<Example>

Find the names of all professors who have a higher salary than some professor in 'CS' department

```
- Select distinct T.name  
  from professor as T, professor as S  
  where T.salary > S.salary and S.deptName = 'CS';
```

- 같은 테이블을 2개의 이름으로 사용하는 대표적인 예제입니다.

6) String Operations

- SQL은 문자열을 지원합니다.

① 일반적인 Substring

= %와 like로 substring 구현

<Example>

```
- Select name  
  from professor  
  where name like '%da%';
```

= professor_table에서 'da'를 포함하는 이름을 갖는 tuple들로 이루어진 결과 테이블을 만듭니다.

② 특수한 Substring

= %기호를 Substring의 기호가 아닌 통상적으로 사용하고 싶습니다.

- escape를 이용합니다.

<Example>

```
- Select cID  
  from course  
  where title like '100W%' escape 'W';
```

- 사용자는 %를 포함한 100%를 title이 포함한 tuple들로 이루어진 결과 테이블을 만들고 싶습니다.
- ₩로 %가 substring과 관련 없다는 표시를 하고 ₩를 escape를 사용하여 지웁니다.

7) Order By

= 결과 테이블의 tuple 순서를 속성 기준으로 정렬합니다.

- 1개의 속성 또는 여러 개의 속성으로 정렬할 수 있습니다.
- 정렬은 default로 오름차순 이고 , desc를 속성 뒤에 붙여서 내림차순으로 정렬 할 수 있습니다.

<Example1>

```
Select distinct name  
from professor  
order by name;
```

= professor table에서 tuple에서 name 속성 선택하고 name을 기준으로 오름차순 정렬을 하여 결과 테이블을 구성합니다.

<Example2>

```
Order by deptName desc, name
```

- 정렬기준이 두개 입니다.
- 먼저 deptname을 기준으로 내림차순 정렬을 하고 , deptname이 같은 tuple은 name을 기준으로 오름차순 정렬을 합니다.

2. INSERT

Insert into course values ('777', 'undecided', 'CS', null);

- = 테이블에 새로운 tuple을 추가합니다.
- = 속성에 알맞은 값을 넣습니다. null도 가능합니다.

3. UPDATE

- = 조건을 만족하는 tuple의 특정 속성을 바꿉니다.

<Example 1_교수님의 연봉이 7000만원 보다 크다면 3%를 인상하고 아니라면 5%를 인상하세요.>

- Update professor // two update statements
set salary = salary*1.03
where salary > 7000;
Update professor // the order is important
set salary = salary*1.05
where salary <= 7000;
- Update professor
set salary = case
when salary <= 7000 then salary*1.05
else salary*1.03
end;

- 연봉(속성)이 7000보다 큰 tuple을 찾고 연봉을 3% 증가시킵니다. 그 후 나머지는 연봉을 5% 증가 시킵니다. 만약 둘의 순서를 바꾸면 , 5%증가를 받은 연봉이 기존에는 7000만원을 넘지 않다가 넘게 되어 오류를 범할 수 있으므로 바꾸지 않습니다. 이런 오류를 예방하기 위해 case / end 구문을 사용합니다.

<Example 2_선택한 학번을 가진 학생의 이수학점(totalCredit)을 업데이트 하세요>

- Update totalCredit values for all students

- Update student S
set S.totalCredit =
 (select sum(credit) // scalar subquery
 from takes natural join course
 where S.sID=takes.sID and grade <> 'F' and grade is
 not null);
- Update student
set totalCredit = 0
where totalCredit is null;

- takes 테이블의 도메인은 학번 , 교과목 이름 , 년도 , 학기 , 학점(A ~ F) 이고 course 테이블의 도메인은 교과목 이름 , 개설학과 , 교과목 타입 , 학점(1시간 ~ 4시간)입니다. 두 테이블을 Natural-Join 하여 student 테이블을 만들어 줍니다.
- student 테이블의 도메인은 학번 , 교과목 이름 , 년도 , 학기 , 학점(A ~ F) , 개설학과 , 교과목 타입 , 학점(1시간 ~ 4시간)입니다. 교과목 이름 이 중복 되므로 학번을 통해 Natural-Join 되었습니다.
- student 테이블에서 선택한 학번의 tuple들을 찾고 학점(A ~ F)와 학점시간(1시간 ~ 4시간)을 찾아서 계산하여 totalCredit을 Update 합니다.
- 학점(A ~ F)의 속성 값이 F인 tuple은 찾지 않습니다.
- totalCredit이 null이면 0으로 업데이트 합니다.

4. DELETE

= 테이블에 존재하는 tuple을 제거합니다.

1) delete from professor

= 교수 테이블의 모든 tuple을 삭제

2) delete from professor where deptName = 'EE'

= deptName(속성)이 EE인 모든 tuple을 삭제합니다.

3)

```
Delete from professor
where deptName in (select deptName
                   from department
                   where building = 'Vision Hall');
```

= 소속되어 있는 학과의 건물이 'Vision Hall'인 tuple을 professor table에서 찾아서 삭제합니다.

4)

```
Delete from professor
where salary < (select avg(salary) from professor);
```

= 봉급이 교수님의 평균 봉급보다 높은 tuple을 professor table에서 찾아서 삭제합니다.

- 어떤 tuple을 삭제하면 다음 tuple의 삭제 조건을 판단할 때 교수님들의 연봉 평균이 달라집니다. tuple을 삭제할 때 마다 교수님들의 연봉 평균을 구하는 일은 귀찮습니다. 그러므로 매번 새로운 평균을 구하지 않고 처음 구한 연봉 평균으로 계속 사용합니다.

5. Duplicates

- SQL 언어는 테이블에서 동일 tuple의 중복을 허용합니다. 즉 , SQL 테이블은 multi-set-tuple로 구성되어 있습니다.
- 관계 대수도 multi-set으로 확장시켜야 합니다.

Suppose multiset relations $r_1(A, B)$ and $r_2(C)$

- $r_1 = \{(1,a) (2,a) (3,b)\}$
- $r_2 = \{(2), (3), (3)\}$
- Then $\Pi_B(r_1)$ is $\{(a), (a), (b)\}$
- $\Pi_B(r_1) \times r_2$ is $\{(a,2), (a,3), (a,3), (a,2), (a,3), (a,3), (b,2), (b,3), (b,3)\}$
- Set 기반에서는 (a,2)의 중복을 제거 했지만 multi-set 기반에서는 중복을 허용합니다.

6. Set Operations(집합 연산)

1) Union

= 합집합(중복 허용 X)

ex) 2009년 1학기 OR 2010년 1학기에 강의한 과목의 ID

2) Intersect

= 교집합(중복 허용 X)

ex) 2009년 1학기 AND 2010년 1학기에 강의한 과목의 ID

3) Except

= except 뒤의 조건은 배제하는 연산(중복 허용 X)

ex) 2009년 1학기에 강의했고 , 2010년 1학기에는 강의하지 않은 과목의 ID

- Union all , Intersect all , Except all처럼 뒤에 all을 붙이면 결과 테이블에 중복을 허용합니다.
- all이 붙지 않으면 중복을 허용하지 않습니다.
 - $m+n$ times in "r union all s"
 - $\min(m, n)$ times in "r intersect all s"
 - $\max(0, m-n)$ times in "r except all s"
- r과 s를 연산했을 때 결과테이블에 생기는 tuple의 개수

<3-4강_Null Values> 10/08

1. null의 정의

= 속성의 값이 null일 수 있습니다.

1) Not Exist

= 처음부터 값이 존재하지 않았습니니다.

2) Unknown

= 처음에는 값이 존재했지만 지금은 존재하는지 모릅니다.

- 대부분 2번째 개념인 Unknown으로 사용합니다.

- null과 관련되면 모호할 때가 많기 때문에 , 해당 상황마다 규칙을 정해 주어야 합니다.

2. null의 규칙

1) 산술식

= null이 포함된 모든 산술식의 결과는 null입니다.

ex) $5 + \text{null} = \text{null}$

2) is null

ex) 'where salary is null'에서 salary 속성의 값이 null이면 True를 결과로 합니다.

3) 비교식

= null과 관련 없이 조건문을 판단할 수 없는 모든 경우(AND TTT , OR FFF)는 unknown으로 결과를 출력합니다.

- where절의 결과는 반드시 True OR False가 나와야 합니다. 그러므로 만약 결과가 unknown이라면 False로 하도록 정의하였습니다.

- OR: $(\text{unknown or true}) = \text{true}$
 $(\text{unknown or false}) = \text{unknown}$
 $(\text{unknown or unknown}) = \text{unknown}$
- AND: $(\text{true and unknown}) = \text{unknown}$
 $(\text{false and unknown}) = \text{false}$
 $(\text{unknown and unknown}) = \text{unknown}$
- NOT: $(\text{not unknown}) = \text{unknown}$

<5-1강_Aggregate Functions> 10/16

*** 3강은 기초적이고 근본적인 SQL 문법이라면 , 5강은 확장된 SQL 문법입니다. ***

*** 4강은 3강을 실제 SQL에서 연습하는 단원이므로 넘어갑니다. ***

1. Aggregate Functions

= 집계 함수

= avg , min , max , sum , count가 존재 합니다.

<Example>

```
Select count(*)  
from student;
```

= student table에 존재하는 모든 tuple의 개수로 이루어진 테이블을 출력합니다.

- 다음 장의 Example을 읽어보면 이 값도 테이블이란 걸 알 수 있습니다.

```
Select avg(salary), max(salary), min(salary)  
from professor  
where deptName= 'CS';
```

= professor table에서 deptName이 'CS'인 속성을 고르고 예시의 함수들의 결과 값으로 이루어진 테이블을 출력합니다.

```
Select count(distinct pID)  
from teaches  
where semester='Spring' and year=2010;
```

= teaches에서 2010년도 봄 학기에 강의를 한 tuple 중에서 교수의 PID의 개수로 이루어진 테이블을 출력합니다.

- distinct 이므로 중복된 것은 세지 않습니다.

2. group by

<Example : professor table에서 deptName(학과)별로 tuple의 개수를 모두 출력하세요.>

```
Select deptName, count(*)  
from professor  
group by deptName;
```

pID	name	deptName	salary
10	Lee	CS	8000
11	Choi	CS	7000
13	Yoon	CS	6000
21	Yang	EE	7000
22	Park	EE	6000
31	Han	Media	9000
32	Paik	Media	6000

deptName	count(*)
CS	3
EE	2
Media	2

- = professor table에서 같은 deptName을 갖는 tuple끼리 그룹을 만들고 각 그룹마다 tuple의 수를 셉니다.
- deptName도 선택해줘서 count의 값이 어떤 학과와 관련되어 있는지 사용자가 알기 쉽도록 해줍니다.
- 연산 결과 속성 2개로 이루어진 테이블을 출력합니다.
- Select 와 from 사이에 나오는 속성은 group by 뒤의 속성에도 있어야합니다. 생략이 가능한 하지만 생략하지 맙시다.

3. Having

= Having을 사용하기 위해서는 반드시 group by가 존재해야 합니다.

= 조건문

<Example1>

```
Select deptName, avg(salary)
from professor
group by deptName
having avg(salary) > 6900;
```

- 각 그룹에서의 모든 avg(salary)를 선택하지 말고 , avg(salary) > 6900을 만족시키는 avg(salary)만을 선택합니다.
- 조건을 담당하는 구문에는 where 와 having이 있습니다. 하지만 having은 반드시 group by와 공존합니다.

<Example2 : 5명 보다 많은 직원으로 구성된 부서에서 부서명과 월급이 4만\$가 넘는 직원의 수로 이루어진 테이블을 출력하세요.>

```
- Select dname, count(*)                               // wrong query
  from department, employee
  where dnumber=dno and salary>40000
  group by dname
  having count(*)>5;

- Select dname, count(*)
  from department, employee
  where dnumber=dno and salary>40000 and
  dno in (select dno from employee
         group by dno
         having count(*)>5)
  group by dname;
```

- 첫 SQL문이 틀린 이유는 Grouping을 할 때 다른 조건을 배제하고 5명 보다 많은 직원을 갖는 tuple을 찾아야 하는데 첫 질의문은 월급이 4만 \$가 넘는 직원이 5명 보다 많은 tuple을 찾습니다.
- 아래의 정답은 중첩 SQL문을 사용하기 때문에 추후에 배웁니다.

4. Null Values

- 기본적으로 모든 연산에서 무시하고 계산합니다. 예외적으로 , Count 연산에서만 무시하지 않고 0으로 계산 합니다.

<Example1>

- 어떤 테이블에서 교수들의 salary가 모두 null이라고 가정합니다.
- select count(salary) = 0을 return 합니다.
- count가 아닌 모든 연산에서는 null을 return 합니다. ex) sum(salary) = null

<Example2>

name	id	hourWage
Kim	100	5000
Lee	200	
Park	300	6000
Yang	400	

- Select count(hourWage) from mytable; // 2
- Select count(distinct hourWage) from mytable; // 2
- Select sum(hourWage) from mytable; // 11,000
- Select hourWage, count(*) from mytable group by hourWage;
// 5000 1
// 6000 1
// null 2
- null 속성으로 이루어진 그룹이 가능합니다.

<5-2강_Join Table> 10/16

1. Join Overview

- = 두 개의 테이블을 조인해서 새로운 테이블을 만드는 연산.
- from 절에서 사용할 수 있습니다.
- 지금까지는 cartesian product 만을 사용했지만 이제는 다양한 Join을 배우겠습니다.
- **Join = Join Type x Join Condition**
- 이번 강의에서는 4개의 Join Type 과 3개의 Join Condition을 배우기 때문에 12개의 Join 연산을 조합해서 사용할 수 있습니다.
- 카타시안 곱으로 하면 결과 table의 크기가 매우 커지고 SQL문 작성이 어려워지므로 보통 natural join을 사용해서 중복을 제거합니다.

2. Join Type

1) Inner Join

= match 되지 않은 tuple은 결과 테이블에 넣지 않습니다.

2) Left Outer Join

= match 되지 않은 tuple 중 왼쪽 테이블에 존재하는 tuple은 결과 테이블에 넣습니다.

3) Right Outer Join

= match 되지 않은 tuple 중 오른쪽 테이블에 존재하는 tuple은 결과 테이블에 넣습니다.

4) Full Outer Join

= match 되지 않은 tuple 중 양쪽 테이블에 존재하는 tuple은 결과 테이블에 넣습니다.

- Outer Join은 Join 당하는 테이블의 정보의 손실을 방지합니다.

3. Join Condition

= Join 당하는 테이블에서 match 되는 속성을 찾는 방법입니다.

1) natural

= 관계 대수에서 배운 Natural Join과 동일합니다.

= 두 테이블에서 겹치는 속성이 있으면 겹치는 속성들을 기준으로 두 테이블을 합칩니다.

- 겹치는 속성은 결과 테이블에서 1개만 존재합니다.

2) on <predicate(=조건문)>

= 두 테이블에 대해서 조건문을 검사했을 때 참이라면 Match 됩니다.

- 겹치는 속성은 결과 테이블에서 모두 존재합니다. 다시 말해서 , A_Table의 속성이 m개 , B_Table의 속성이 n개라면 On 으로 합쳐진 C_Table의 속성은 m+n개입니다.

3) using (A1, ... , An)

= A1 ~ An은 겹치는 속성이고 , 그 속성들을 기준으로 두 테이블을 합칩니다.

- natural 과 유사합니다.

- 겹치는 속성은 결과 테이블에서 1개만 존재합니다.

4. Join Operation Example

= natural은 join type 보다 앞에 오고 , on 과 using은 join type 보다 뒤에 옵니다.

- 예시에 적은 구문이 from 절에 들어갑니다.

<Join 당할 2개의 테이블>

Relation myCourse

cID	title	deptName	credit
BIO-301	Genetics	Biology	4
CS-301	DB	CS	4
CS-302	AI	CS	3

Relation myPrereq

cID	prereqCID
BIO-301	BIO-101
CS-301	CS-101
CS-303	CS-101

1) Right Outer + natural

- myCourse natural right outer join myPrereq

cID	title	deptName	credit	prereqCID
BIO-301	Genetics	Biology	4	BIO-101
CS-301	DB	CS	4	CS-101
CS-303	null	null	null	CS-101

- natural 이므로 겹치는 속성인 cID 기준으로 합칩니다.
- right outer 이므로 오른쪽 테이블인 myPrereq에서 겹치지 않은 CS-303 tuple도 Join 하여 포함시킵니다.

2) Left Outer + natural

myCourse natural left outer join myPrereq

cID	title	deptName	credit	prereqCID
BIO-301	Genetics	Biology	4	BIO-101
CS-301	DB	CS	4	CS-101
CS-302	AI	CS	3	null

- Right Outer + natural과 같은 원리

3) Full Outer + natural

• myCourse natural full outer join myPrereq

cID	title	deptName	credit	prereqCID
BIO-301	Genetics	Biology	4	BIO-101
CS-301	DB	CS	4	CS-101
CS-302	AI	CS	3	null
CS-303	null	null	null	CS-101

- Right Outer + natural과 같은 원리

4) Inner Join + On

- myCourse inner join myPrereq on myCourse.cID = myPrereq.cID

myCourse.cID	title	deptName	credit	myPrereq.cID	prereqCID
BIO-301	Genetics	Biology	4	BIO-301	BIO-101
CS-301	DB	CS	4	CS-301	CS-101

- On의 조건이 myCourse.cID = myPrereq.cID입니다.
- Inner Join이므로 On의 조건을 만족하는 tuple 끼리 Join을 합니다.
- On이므로 중복된 속성도 결과 테이블에 모두 적어줍니다.

5) Left Outer Join + On

- myCourse left outer join myPrereq on myCourse.cID = myPrereq.cID

myCourse.cID	title	deptName	credit	myPrereq.cID	prereqCID
BIO-301	Genetics	Biology	4	BIO-301	BIO-101
CS-301	DB	CS	4	CS-301	CS-101
CS-302	AI	CS	3	null	null

- 위의 Inner Join + On과 방법은 똑같으나 이 경우에는 Left Outer Join을 해야 합니다.

6) Left Outer Join + Using

myCourse left outer join myPrereq using(cID)

cID	title	deptName	credit	prereqCID
BIO-301	Genetics	Biology	4	BIO-101
CS-301	DB	CS	4	CS-101
CS-302	AI	CS	3	null

- Using은 natural과 거의 동일하므로 매개변수의 속성이 natural에서 겹치는 속성으로 봐도 무방합니다.
- cID는 두 테이블에서 겹치는 속성이고, cID를 기준으로 natural + Left Outer Join을 합니다.

<5-3강_Nested Sub-Query> 10/16 ~ 10/19

1. Nested Sub-Query

- = 중첩 서브 질의
- Sub-Query의 결과도 Table입니다.
- 보통 Where절 또는 From절에 존재합니다.

<Single-Row Sub-Query>

- = Sub-Query의 결과로 오직 1개의 tuple과 1개의 속성만 갖는 경우입니다.
- Scalar Sub-Query라고도 합니다.

<Example : 'CS'학과에서 최고의 연봉을 받는 교수의 이름을 구하세요.>

```
Select name  
from professor  
where salary = (select max(salary)  
                from professor  
                where deptName='CS');
```

= Where절에서 salary는 value이고 (Sub-Query)는 table이므로 원래는 비교할 수 없으나 , 이 경우에는 결과 테이블에 tuple이 1개이고 그 tuple의 속성이 1개이므로 특수하게 비교할 수 있습니다.

2. Sub-Query Operator

1) IN

<Example : 교수의 PID가 10 , 21 , 22 중 하나인 tuple을 선택하시오.>

```
Select name, salary
from professor
where pID in (10, 21, 22);
```

```
Select name, salary
from professor
where pID=10 or pID=21 or pID=22;
```

- = Sub-Query에 주어진 속성 값에 해당하는 집합 (10, 21, 22)을 만들고 pID가 그 집합에 존재하면 Where 절에서 True입니다.
- SQL에서 집합은 ()로 표현할 수 있습니다.

```
Select count(distinct sID)
from takes
where (cID, semester, year) in (select cID, semester, year
                                from teaches
                                where pID= 10);
```

교수의 PID가 10인 교수가 가르친 과목을 들은 학생의 수를 구하고 싶을 때 -> 학생의 년도 학기 , cid가 필요.

cid semester year를 teaches 테이블에서 pid가 10일 때 속성을 뽑아서 테이블 만들고 , 어떤 tuple이 그 조건을 만족시키면(where절)

2) NOT IN

- = IN과 형식은 똑같으나 , 집합에 존재하지 않으면 Where 절에서 True입니다.

3) Comparison

= SQL에서는 값과 집합의 비교가 가능합니다.

① some

= 집합 중에서 어떤 원소 하나라도 조건을 만족하면 Where 절에서 True입니다.

<Example 1>

$(5 < \text{some}\{0,5,6\}) = \text{True}$

= 0과 5는 5보다 크지 않으나, 6이 5보다 큼니다. 6이 조건을 만족시켰으므로 True입니다.

<Example 2 : 'CS'학과의 어떤 교수가 최소한 다른 교수 1명 보다 연봉을 많이 받으면 그 교수의 이름을 뽑아서 테이블을 구하시오.>

```
Select distinct T.name  
from professor as T, professor as S  
where T.salary > S.salary and S.deptName='CS';
```

```
Select name  
from professor  
where salary > some (select salary  
                     from professor  
                     where deptName='CS');
```

② all

= 집합 중에서 모든 원소가 조건을 만족하면 Where 절에서 True입니다.

<Example 1>

$(5 < \text{all}\{0,5,6\}) = \text{False}$

= 6이 5보다 크지만 0과 5는 5보다 크지 않습니다. 모든 원소가 조건을 만족시키지 않았으므로 False입니다.

<Example 2 : 'CS'학과의 어떤 교수가 다른 모든 교수 보다 연봉을 많이 받으면 그 교수의 이름을 뽑아서 테이블을 구하시오.>

```
Select name
from professor
where salary > all (select salary
                    from professor
                    where deptName='CS');
```

3. Correlated Sub-Query

= Sub-Query가 Outer-Query(큰 Query)의 변수를 Where 절에서 사용하는 SQL문

- 시간이 많이 걸립니다.

1) exists

= Sub-Query에 tuple이 존재하면 True , 존재하지 않으면 False

<Example : 2009년 가을과 2010년 가을에 모두 개설된 과목을 구하시오>

```
Select S.cID
from teaches as S
where S.semester = 'Fall' and S.year= 2009 and
exists (select *
        from teaches as T
        where T.semester = 'Fall' and T.year= 2010
        and S.cID = T.cID);
```

- 2009년 가을을 찾는 것은 매우 간단하지만 2010년 가을을 찾을 때는 2009년 가을에서 찾은 과목과 같은 cID를 갖는 과목을 찾아야 합니다.

- 그러므로 Outer-Query의 S.cID를 사용하여 같은지 비교합니다. 이렇게 하면 쉽게 같은 과목을 찾을 수 있습니다.

- exists의 경우 Sub-Query의 조건에 맞는 tuple이 1개 이상 존재하면 True입니다. 따라서 2010년 검사에서 2009년과 같은 cID의 과목이 존재하면 exist는 True이고 존재하지 않으면 False 입니다.

- And TTT를 생각해보면 Where절에서 Sub-Query가 False면 조건문은 반드시 False이므로 훌륭한 SQL문 입니다.

2) not exists + except

```
Select S.sID, S.name
from student as S
where not exists ( (select cID
                    from course
                    where deptName = 'CS')
except (select T.cID
        from takes as T
        where S.sID = T.sID) );
```

= cs 학과에서 개설된 모든 교과목을 수강한 학생 찾기.

= student table과 takes table course table이 있는데 , s tuple마다 학생들을 검사하는데 , 이게 공집합이면 not exist라 참.

- 이거 모르겠어.

3) unique

= Sub-Query에 중복이 없으면 True.

- unique(공집합) = True , unique(null , null) = True -> null은 알 수 없는 값이므로 중복이 아니라고 생각합니다.

<Example : 2009년이 총 4학기라고 할 때 , 한 번 또는 단 한 번도 개설되지 않은 과목을 찾으시오>

```
Select C.cID
from course as C
where unique (select T.cID
               from teaches as T
               where C.cID=T.cID and T.year=2009);
```

- 2009년에 개설이 2번 된 강좌라면 , Sub-Query를 검사할 때 T.cID가 2개 이상 결과 테이블에 존재하므로 중복이 생깁니다. 그 결과 False를 Return 합니다.
- 개설이 0번 OR 1번 된 강좌라면 , 테이블에 없거나 1개가 존재하므로 True를 Return 합니다.

4. From절에 올수 있는 Sub-Query

<Example : avgSalary가 6900보다 큰 교수의 학과와 avgSalary를 구하시오>

```
Select deptName, avgSalary
from (select deptName, avg(salary) as avgSalary
      from professor
      group by deptName)
where avgSalary > 6900;
```

- = From 절에서 탐색할 Table을 Sub-Query로 구합니다.
- From 절에서 group by를 하므로 having을 쓰지 않습니다.

5. lateral

- = 이런 것이 있다고만 알아둡시다.

6. with

- = SQL문의 연산을 쉽게 하기 위해 미리 사용자 지정 테이블을 만들고 , 그 테이블을 SQL에서 사용합니다.

<Example : 가장 큰 예산을 갖는 학과를 구하시오>

```
With      maxBudget(value) as
          (select max(budget)
           from department)
select deptName, budget
from department, maxBudget
where department.budget = maxBudget.value;
```

<5-4강_Ranking>

- 새로운 내용을 듣고 싶으므로 나중에 듣자!
- 시험 끝나고 삭제

<6-1강_View> 10/29

1. View의 정의

= DB에 저장되어 있는 실제 Table(=Actual Table = Base Table)에서 특정 속성을 제거한 가상 Table(=virtual Table)

= User_1이 Professor_table을 만들었는데 , Professor_table을 User_2도 접근할 수 있게 해주고 싶습니다. 하지만 User_2가 Professor_table의 salary 속성은 접근할 수 없어야 합니다. 이 때 Professor_table(=Base Table)에서 salary 속성을 제외하고 Select 하여 View를 만들어서 User_2가 View를 사용하게 합니다. 특정 속성을 감추기 때문에 **보안**과 직결된 개념입니다.

- View는 Virtual Table이므로 DB에 실제로 저장되어 있지 않습니다.

- View는 Virtual Table이지만 결국에는 Table이므로 **SQL Query에서 Table이 올 수 있는 자리에 View가 올 수 있습니다.** 그러므로 from 절에 다른 view를 사용해서 새로운 view를 만들 수 있습니다.

- View는 Base Table의 일부분(부분집합)이기 때문에 **View는 언제나 최신의 데이터**를 갖고 있습니다.

2. View 생성

= Create view 'view 이름' as 'Actual Table'

3. View Expansion

= View 정의문에 다른 view가 들어가 있을 때 데이터베이스는 이를 치환하여 해석합니다.

<Example>

```
Create view myFaculty as
  select plD, name, deptName
  from professor
  where salary > 50000;
```

```
Create view myFacultyCS as
  select plD, name
  from myFaculty
  where deptName = CS;
```

The above view can be expanded:

```
Create view myFacultyCS as
  select plD, name
  from professor
  where deptName = 'CS' and salary>50000;
```

= myFacultyCS는 다른 View인 myFaculty를 Base Table로 사용합니다. 그러므로 myFaculty를 치환하여 3번째 Create 문처럼 해석합니다.

4. View Modifications

= Table을 변경하는 3대 함수(insert , update , delete)를 4가지 제약 조건만 만족하면 View에서도 할 수 있습니다.

1) 제약 조건

① Base Table은 반드시 1개여야 합니다.

```
Create view professorInfo as
select pID, name, building
from professor, department
where professor.deptName= department.deptName;

Insert into professorInfo values ('2345', 'White', 'Vision Hall');
```

- 2개가 오면 Insert의 경우 , 'White'를 어떤 table의 name 값으로 넣어야 할지 알 수 없습니다.

② 집계함수가 포함되어 있으면 안 됩니다.

```
- Create view departmentTotalSalary(deptName, totalSalary) as
select deptName, sum(salary)
from professor
group by deptName;
```

```
Insert into departmentTotalSalary values ('CS', 100,000);
```

- 집계함수에 대한 갱신을 베이스 테이블에 반영하는 방법이 없습니다.

③ Group by 와 having 절이 존재하면 안 됩니다.

④ 잘못된 속성에 대한 데이터를 넣으면 안 됩니다.

= with check option

Create view CSProfessor as

select *

from professor where deptName= 'CS';

Insert into CSProfessor values ('255', 'Brown', 'EE', 100000);

2) Example

Insert into myProfessor values ('12345', 'Lee', 'CS');

This insertion causes the tuple ("12345', 'Lee', 'CS', null)
to be inserted into the "professor" relation

- myProfessor(View)는 3가지 속성으로 구성되어 있고 , Professor(Base Table)는 4가지 속성으로 구성되어 있습니다. 이런 경우 해당 속성에 null을 넣어서 Insert 해줍니다.
- View에 Insert를 하는 질의문은 결국 Base Table에 Insert 하는 것입니다. View는 Virtual Table이기 때문입니다.

<6-2강_Integrity Constraints(=IC)> 10/30

1. Integrity Constraints

= 무결성 제약 조건

= DBMS에서 항상 만족시켜야 하는 제약 조건.

- DB에 존재하는 Table의 제약조건 , Column의 제약조건 , Tuple의 제약조건을 모두 만들어야 의미 있는 DB가 됩니다.

- **Accuracy(정확성)**와 **Consistency(일치성)**를 보장하기 위해서 존재합니다.

- 무결성을 지키지 않은 tuple이나 속성이 존재한다면 DBMS는 사용자에게 ERROR를 Return 합니다.

- 단일 테이블과 다수의 테이블 마다 지켜야 할 제약 조건의 종류가 다릅니다.

ex) 박사과정 학생은 지도교수가 있어야합니다.

ex) 최저시급은 8,590원을 넘어야합니다.

2. 단일 테이블에서의 4가지 Integrity Constraints

1) not null

= 테이블의 속성 값이 null이면 안 됩니다.

- 해당 속성 옆에 적어줍니다.

```
name varchar(20) not null
budget numeric(12,2) not null
```

2) primary key (=Entity 무결성)

= Primary key는 중복되는 값을 가지면 안 됩니다.

= **Primary key로 지정된 속성은 어떤 값도 null이 되면 안 됩니다.**

- 해당 속성 옆에 적어줍니다. 테이블 그림에서는 밑줄을 쳐서 표시합니다.

3) unique

- = Candidate key는 중복되는 값을 가지면 안 됩니다.
- = Candidate key로 지정된 속성은 null 값이 가능 합니다.

4) check (P), where P is a predicate

- = P는 조건이며 , 어떤 속성은 조건 P를 만족하는지 검사합니다.
- 기존에 Where절에 들어 갈 수 있었던 문장은 모두 들어갈 수 있지만 , Sub-Query는 불가능 합니다.

<Example : 학기가 (봄 , 여름 , 가을 , 겨울)에 포함되어 있는지 검사합니다.>

```
Create table teaches (  
    plD          char(5),  
    clD          char(5),  
    semester     varchar(10),  
    year         numeric(4,0),  
    classroom     char(5),  
    primary key (plD, clD, semester, year),  
    check (semester in ('Spring', 'Summer', 'Fall', 'Winter') ) -  
);
```

3. Foreign Key(외래키)

= 서로 다른 두개의 테이블을 연결해주는 연결 다리 역할을 하는 Primary Key

- 하나의 table만 사용해서 데이터를 분석할 때는 큰 의미가 없지만 두 개 이상의 table을 조합해서 데이터를 분석할 때에는 중요한 개념입니다.
- SQL에서 가장 주의해야 할 일은 tuple의 중복입니다. 다양한 속성을 가진 tuple에서 1개의 속성만이 바뀐 tuple을 계속 추가하다 보면 다른 속성들은 모두 중복이 되어서 Table의 크기가 엄청 커지게 되고 , 공간의 낭비가 생깁니다. 이를 방지하게 위해 table을 쪼개서 두 개의 table로 저장을 하고 필요할 때 마다 참조를 합니다.
- 참조를 할 때 참조의 매개 Key가 다양하면 안 됩니다. 그러므로 참조의 매개는 반드시 Primary Key로 합니다.

<Example_1>

teaches		
pID	cID	...
100	CS101	...
100	CS201	...
200	EE101	...

professor		
<u>pID</u>	name	...
100	Kim	...
200	Lee	...

```
Create table teaches (  
  pID      varchar(5),  
  cID      varchar(5),  
  ...  
  primary key(pID, cID, semester, year)  
  foreign key(pID) references professor,  
  foreign key(cID) references course);
```

- teaches의 속성 pID는 반드시 professor 테이블에 존재해야 하므로 pID는 foreign key입니다.
- teaches의 속성 cID는 반드시 course 테이블에 존재해야 하므로 cID는 foreign key입니다.

<Example_2 : 쇼핑물> [참고 사이트 : https://brunch.co.kr/@dan-kim/26](https://brunch.co.kr/@dan-kim/26)

user_id	user_pwd	name	gender	address	order_date	product_id	product_title	qty
marco117	xxx	Marco	male	451 Sierra St.	2019-10-01	p131	coke	5
marco117	xxx	Marco	male	451 Sierra St.	2019-10-01	p132	sprite	10
marco117	xxx	Marco	male	451 Sierra St.	2019-10-02	p131	coke	1
marco117	xxx	Marco	male	451 Sierra St.	2019-10-02	p132	sprite	5

= Marco117의 아이디를 가진 사람이 물건을 계속 주문하면 붉은색으로 표기된 속성들은 중복되어서 table을 채우는 낭비를 발생시킵니다.

order_d...	product_id	product_title	qty	user_id
2019-10-01	p131	coke	5	marco171
2019-10-01	p132	sprite	10	marco171
2019-10-02	p131	coke	1	marco171
2019-10-02	p132	sprite	5	marco171

user_id	user_pwd	name	gender	address
marco117	xxx	Marco	male	451 Sierra St.

= 2개의 테이블로 구분하고 , Primary Key인 user_id를 Foreign Key로 설정하여 참조하도록 합니다.

4. 두 개 이상의 테이블에서의 Integrity Constraints : Referential integrity constraint(참조 무결성 제약)

= 두 테이블에서 어떤 속성을 foreign key로 설정하면 , 자식 테이블에서 해당 foreign key의 값을 변경하는 경우 부모 테이블의 제약을 받게 되는 현상

- 두 테이블 이상에서의 제약 관계입니다.
- Foreign Key 제약이라고도 부릅니다.
- 어떠한 경우에도 참조 무결성 제약은 지켜져야 합니다.

teaches		
pID	cID	...
100	CS101	...
100	CS201	...
200	EE101	...

professor		
<u>pID</u>	name	...
100	Kim	...
200	Lee	...

```
Create table teaches (  
  pID      varchar(5),  
  cID      varchar(5),  
  ...  
  foreign key(pID) references professor,  
    on delete cascade,  
    on update cascade,  
  ...  
);
```

- cascade : 연쇄적

*** 그림을 활용하여 on delete 와 on update를 다음 장에서 설명하겠습니다. ***

1) on delete

<Question : professor에서 pID가 100인 tuple을 삭제한다면 어떻게 해야 하는가?>

① on delete만 존재

= teaches에는 pID가 100인 tuple이 있고 professor에는 pID가 100인 tuple이 없기 때문에 참조 무결성 제약이 깨지므로 지울 수 없다는 ERROR 메시지를 출력합니다.

② on delete cascade

= teaches에 pID가 100인 tuple을 연쇄적으로 모두 삭제 합니다.

③ on delete set null

= teaches에 pID가 100인 tuple의 pID를 null로 바꿉니다.

- 만약 teaches의 pID가 primary key라면 , primary key는 값으로 null을 허용하지 않으므로 지울 수 없다는 ERROR 메시지를 출력합니다.

<Question : teaches에서 pID가 100인 tuple을 삭제한다면 어떻게 해야 하는가?>

= 참조 무결성 제약이 깨지므로 해당 액션은 허용하지 않습니다.

2) on update

<Question : **professor**에서 pID가 100인 tuple을 400으로 업데이트 한다면 어떻게 해야 하는가?>

① on update만 존재

= teaches에는 pID가 400인 tuple이 있고 professor에는 pID가 400인 tuple이 없기 때문에 참조 무결성 제약이 깨지므로 지울 수 없다는 ERROR 메시지를 출력합니다.

② on update cascade

= teaches에 pID가 100인 tuple을 연쇄적으로 400으로 업데이트 합니다.

③ on update set null

= teaches에 pID가 100인 tuple의 pID를 null로 바꿉니다.

- 만약 pID가 primary key라면 , primary key는 값으로 null을 허용하지 않으므로 지울 수 없다는 ERROR 메시지를 출력합니다.

<Question : **teaches**에서 pID가 100인 tuple을 400으로 업데이트 한다면 어떻게 해야 하는가?>

= 참조 무결성 제약이 깨지므로 해당 액션은 허용하지 않습니다.

4. Insert Tuple

```
Create table person (  
    ID          char(10) primary key,  
    name        char(40),  
    mother      char(10),  
    father      char(10),  
    foreign key (mother) references person,  
    foreign key (father) references person);
```

- name이 a고 mother가 b고 father가 c인 tuple을 Insert 하기 위해서는 b와 c에 관련된 tuple이 먼저 Table에 존재해야 합니다. 참조 무결성 때문에 언제나 해당 tuple을 만들 때 이를 검사해야 하며 매우 복잡하고 불편합니다. 이를 해결하기 위한 2가지 방법이 존재합니다.

1) name이 a인 tuple을 Insert 할 때 mother 와 father를 null로 합니다. 참조 무결성은 만족하지만 추후에 모든 tuple에 대해 update를 해야 하므로 비용이 매우 큽니다.

2) Deferred(연기)

= 참조 무결성 제약을 tuple을 Insert 할 때 검사 하는 것이 default 이지만 나중에 할 수 있게 해줍니다.

5. DBMS에서 허용하지 않는 integrity constraints

1) Sub-Query

= check(P)문에서 P에 Sub-Query를 사용하고 싶지만 비용이 크기 때문에 불가능합니다.

2) Assertion(보장)

```
Create assertion myVerifyTotalCredit check
(not exists
  (select s1.sID
   from student s1
   where s1.totalCredit <> (select sum(credit)
                             from takes, course
                             where s1.sID = sID
                             and course.cID=takes.cID
                             and grade is not null
                             and grade <> 'F')
  )
);
```

- TotalCredit은 학생이 지금까지 수강한 강좌의 이수시간의 총 합.

- TotalCredit과 다른 테이블에서 수강한 강좌의 이수학점을 모두 더했을 때(F는 제외) 합이 같도록 언제나 **보장**을 하고 싶습니다.

- 보장을 위해 assertion으로 조건을 반드시 만족하도록 DBMS에게 검사를 시키고 싶지만 이는 비용이 매우 크므로 DBMS에서 허용하지 않습니다.

<6-3강_Trigger> 11/01

- Trigger를 사용하지 않았을 때의 DBMS는 insert update delete 같이 DB의 데이터를 변경하는 작업을 수행하면 그 때 마다 전체 tuple을 검사해서 무결성을 지켜야 했습니다. 그 결과 매우 큰 비용이 생기게 되었습니다.
- Trigger는 데이터의 변경이 발생하였을 때 DB에서 자동으로 특정 조건에 맞는 tuple의 무결성을 검사합니다. 그 결과 비용이 크게 줄어들었습니다.

1. Trigger

= Event -> Condition -> Action (ECA) 구조를 갖는 모델이며 , 이벤트가 발생했을 때 조건이 맞으면 액션을 취합니다.

1) Trigger에서는 변수 사용이 가능합니다.

ex) referencing old tuple as = 삭제 , 업데이트 전의 tuple에 대한 변수

ex) referencing new tuple as = insert , update 이후의 tuple에 대한 변수

2) Event

= insert , delete , update 같이 DB를 변경하는 작업.

3) Condition

= 조건

4) Action

= DB에 필요한 작업을 수행합니다.

2. Row Trigger(행 트리거)

= Tuple 단위로 변경사항을 검사합니다.

1) 학점 변경 기간에 어떤 학생의 어떤 과목의 grades가 F -> A+로 변경이 일어나서 학생의 total credits가 변경되는 경우

```
Create trigger myCred after update of grade on takes
referencing new row as nrow
referencing old row as orow
for each row
when nrow.grade <> 'F' and nrow.grade is not null
    and (orow.grade = 'F' or orow.grade is null)
begin
    Update student
    set totalCredit = totalCredit +
        (select credit
         from course
         where cID = nrow.cID)
    where sID = nrow.sID;
end;
```

<변수 선언>

= 'referencing new row as nrow'

<Event>

= after update of grade on takes'

= takes table에 grade가 변경되었습니다.

<Condition>

= when절

= old grade 가 'F' OR null 이고 , new grade가 'F' OR' null이 아닐 때

<Action>

= begin ~ end

= student table에 totalCredit을 업데이트 합니다.

<for each row>

= tuple 단위로 trigger를 하겠습니다.

2) 은행 계좌의 마이너스 통장은 어떻게 생성 되는가?

```
account(aNumber, balance)
loan(lNumber, amount)
depositor(cName, aNumber)
borrower(cName, lNumber)
```

- aNumber = 예금 계좌번호
- lNumber = 대출 계좌번호
- account = 계좌 , balance = 예금한 금액
- loan = 대출 , amount = 대출한 금액
- depositor = 고객에 갖고 있는 예금 계좌
- borrower = 고객이 갖고 있는 대출 계좌

```
Create trigger myOverdraft after update on account
referencing new row as nrow
for each row
when nrow.balance < 0
begin atomic
    Insert into borrower
        (select cName, aNumber
         from depositor
         where nrow.aNumber = depositor.aNumber);
    Insert into loan values (nrow.aNumber, -nrow.balance);
    Update account set balance = 0
        where account.aNumber = nrow.aNumber;
end
```

<변수 선언>

= 'referencing new row as nrow'

<Event>

= after update of balance on account

<Condition>

= when절

<Action>

= begin ~ end

- borrower에 추가

- loan에 tuple 추가

- account에 balance = 0으로 업데이트

- balance < 0 이 되는 경우에 balance 값을 음수로 세팅하지 않고 , 대출 계좌를 새로 생성하여 대출 계좌의 amount를 그만큼 증가시킵니다.
- 예금 계좌의 balance는 0으로 만들어줍니다.

3. Event 발생 이전의 Trigger

= 앞에서는 Trigger가 Event 발생 이후에 수행되었지만 , Event 발생 이전에도 수행될 수 있습니다.

<Example : 사전 검사>

= 학기말에 학생이 수강한 과목에 대한 학점을 부여해야 하는데 그 작업을 하기 전에 학점이 blank 상태라면 null 값을 미리 부여해 놓습니다.

```
Create trigger mySetNull before update on takes  
referencing new row as nrow  
for each row  
when (nrow.grade = ' ' )  
Update takes set nrow.grade = null;
```

4. Statement Level Triggers

= **Table** 단위로 변경 사항을 검사합니다.

- 매우 많은 Tuple 들이 변경 되는 경우 , Tuple 단위로 검사하는 것 보다 Table 전체를 검사해서 한 번에 처리하는 것이 효율적입니다.
- 변수 지정도 Table 단위이므로 old_table , new_table로 사용됩니다.

<Example : 어떤 회사에서 많은 수의 직원들의 월급을 Update하고 직원들 월급의 총 합인 totalSalary를 구해야 합니다.>

```
Create trigger myTotalSalaryStateLevel
after update of salary on employee
referencing old table as O
referencing new table as N
for each statement
when exists(select * from N where N.dnumber is not null) or
      exists(select * from O where O.dnumber is not null)
Update department as D
set D.totalSalary = D.totalSalary
+ (select sum(N.salary) from N where D.dno=N.dnumber)
- (select sum(O.salary) from O where D.dno=O.dnumber)
where D.dno in ( (select dnumber from N) union
                (select dnumber from O) );
```

- 테이블 단위로 업데이트를 해서 1번만 업데이트를 합니다.

*** 자세한 내용을 이해하지 못했습니다. ***

*** chap6 (5)강 6분 ~ 10분 ***

5. Trigger를 사용하는 경우

*** 교양처럼 듣습니다. ***

- 요약 데이터를 만들 때
- 원본 데이터 중에 극히 일부가 변경 되었을 때 복사본에 대해서도 무결성을 위해 변경을 해주어야 합니다.
- Trigger에 체인을 걸어서 의도하지 않은 변경을 방지합니다.

<6-4강_Table Authorization> 11/05

- Table과 view에 대해서 권한을 주어야함.
- DBA는 모든 권한을 갖고 있습니다.

1. Privileges in SQL

1) select

= 어떤 relation을 읽을 수 있도록 권한을 부여합니다.

2) insert , update , delete

= 각각 tuple을 insert, update , delete 할 수 있게 권한을 부여합니다.

3) reference

= foreign key와 관련이 있습니다. 참조를 할 수 있게 권한을 부여해서 Foreign key를 지정할 수 있게 해줍니다.

4) usage

= 도메인을 사용할 수 있게 권한을 부여합니다.

5) all privilege

= 모든 권한을 부여 합니다.

2. Grant

= 다른 유저에게 권한을 부여합니다.

Grant <privilege list> on <relation name or view name> to
<user list> [with grant option]

<user list> can be a user-id, a role, or "public" which means all valid users

- 어떤 relation이나 view에 대한 특정 권한(privilege list)을 다른 유저에게 부여합니다.
- user list에는 user_id , role , public(모든 유저)이 올 수 있습니다.
- with grant option : 권한을 받는 유저가 또 다른 유저에게 권한을 줄 수 있게 됩니다.

<Example>

Grant select on professor to U1, U2, U3;

Grant select on professor to U4 with grant option;

- Gives U4 the select privileges on "professor" and allows U4 to grant this privilege to others

Grant references (deptName) on department to Lee;

- Gives Lee a privilege to create a foreign key referencing department(deptName)
- Why we need to have this?

- 1) professor_table의 select에 대한 권한을 u1 u2 u3에게 줍니다.
- 2) u4에게 select에 대한 권한을 또 다른 유저에게 줄 수 있는 권한을 줍니다.
- 3) department의 deptName을 참조할 수 있는 권한을 Lee에게 줍니다. Lee는 deptName을 참조할 수 있으며 foreign key를 만들 수 있게 됩니다.

3. Revoke

= 어떤 유저로부터 테이블에 대한 특정 권한을 뺏습니다.

```
Revoke <privilege list> on <relation name or view name>  
from <user list> [restrict | cascade];
```

1) Cascade

```
Revoke select on professor from U1, U2, U3 cascade;
```

- = 권한 부여가 U1 -> U2 -> U3일 때 U1이 U2에게서 권한을 뺏으면 연쇄적으로 U3도 권한이 뺏깁니다.
- Default 입니다.

2) Restrict

```
Revoke select on professor from U1, U2, U3 restrict;
```

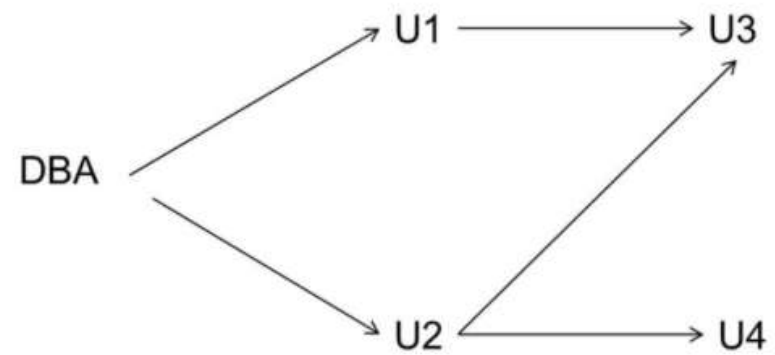
- = 뜻하지 않게 연쇄적으로 권한을 뺏는 것을 방지하고 싶을 때 사용합니다.
- = 연쇄적으로 권한을 회수시킨다면 DBMS 에서 스스로 명령어를 Fail 시킵니다.

3) 권한_X를 2명 이상이 한 명의 유저에게 주었을 때

- = 만약 u2가 u1에게 권한_X를 받았고 , u4에게도 권한_X를 받았다고 가정해봅시다. u1이 u2에게서 권한_X를 Revoke해도 u2는 u4에게서 받은 권한이 있기 때문에 계속 권한_X를 갖고 있습니다.

4. 권한 관리

= 그래프를 통해 관리합니다.



- 권한을 주는 유저 : 화살표를 쏩니다.
- 권한을 받는 유저 : 화살표를 받습니다.

<6-5강_View Authorization> 11/07

1. View_Authorization

= View는 Base_Table의 일부분으로 구성되어 있습니다. 그러므로 View를 제작하기 위해서는 최소한 Base_Table에 대해 최소한 Select 권한이 있어야 합니다. 다시 말해, View를 만들었다면 Base_Table에 대해 Select 권한이 있다고 해석할 수 있습니다.

- 만약 View에 대해 Insert, Update, Delete 권한을 갖고 싶다면, Base_table로 부터 각각 Insert, Update, Delete 권한이 있어야 합니다.
- View를 생성하기 위해서 Resource에 대한 권한은 필요하지 않습니다.

<Example_1 : 교수님이 2015년 가을학기에 존재한 강의와, 해당 강의를 가르친 교수를 조교에게 알려주고 싶지만 교수의 연봉은 숨기고 싶다.>

```
Create view myTeach as
select    name, title
from      professor, teaches, course
where teaches.plD=professor.plD and course.clD=teaches.clD
and semester='Fall' and year=2015;
```

= 조교에게 알려줄 정보만 담은 View를 만들고, 그 View에 대한 권한을 조교에게 줍니다.

- 카타시안 곱으로 만든 테이블 집합에서 name 과 title 만 선택하여서 MyTeach라는 View를 만듭니다. 그 View에 대한 권한을 조교에게 전해줍니다.
- 조교는 myTeach에 있는 모든 데이터를 읽을 수 있으나 Professor, Course, Teaches에는 직접 접근하지 못하므로, 데이터를 읽을 수 없습니다.
- 'Select * from myTeach'를 조교가 사용하면 DBMS는 MyTeach(view)에 대한 권한만을 체크합니다. Professor, Teaches, Course에 대한 권한을 체크하지 않습니다.

<Example_2>

```
user1>Create view CSProfessor as
      (select * from professor where deptName = 'CS');
user1>Grant select on CSProfessor to staff;
staff>Select * from CSProfessor;
```

- User1은 CSProfessor(view)를 만듭니다.
- View를 만들었기 때문에 Base table인 professor에서 최소한 select 권한을 갖고 있음을 의미합니다.
- User1은 Staff(다른 유저)에게 CSprofessor에 대한 권한을 주고 , 그로 인해 Staff는 CSprofessor에서 Select를 할 수 있게 됩니다.
- Staff가 Professor에 대한 권한이 없는 것은 아무런 문제가 되지 않습니다. 왜냐하면 DBMS는 CSProfessor에 대한 권한만을 체크하기 때문입니다.
- User1이 Professor에 대한 Update 권한이 없는 것은 아무런 문제가 되지 않습니다. 왜냐하면 View를 만들기 위해서는 Professor에 대해 Select를 할 수 있는 권한만 존재하면 되기 때문입니다.

2. Roles

= 어떤 유저가 받은 권한들의 집합

- 자료형이 권한인 **Vector**라고 이해해도 괜찮습니다.
- 다른 유저들에게 Role을 전해줄 수 있습니다.

<Example>

```
Create role teller;  
Create role manager;  
Grant select on branch to teller;  
Grant update(balance) on account to teller;  
Grant all privileges on account to manager;  
  
Grant teller to manager;  
  
Grant teller to Kim, Park;  
Grant manager to Lee;
```

- Revoke는 Grant와 동일한 방식으로 사용합니다.

- teller , manager : Role

- ① branch_table의 select 권한을 teller에게 추가합니다.
- ② account_table의 update 권한을 teller에게 추가합니다.
- ③ account_table의 모든 권한을 manager에게 추가합니다.

- teller는 ①,②로 인해 branch의 select 권한 + account의 update 권한을 갖습니다.

- manager는 ③과 Grant teller to manager(teller가 갖는 권한을 manager에게 줌)로 인해 teller가 갖고 있는 모든 권한과 account의 모든 권한을 갖습니다.

- Teller가 갖고 있는 권한들을 Kim , Park에게 전해줍니다.

- Manager가 갖고 있는 권한들을 Lee에게 전해줍니다.

3. SQL Authorization의 한계

- SQL에서는 Tuple level에 대한 권한 부여/회수가 없습니다. 언제나 Table 단위입니다.
- Web DB에서는 tuple 단위로 권한 부여/회수가 가능합니다.

*** 6-5강은 재귀적 Query에 관한 내용입니다. SQL이 재귀적 Query를 사용할 수 있다는 정도만 알고 넘어갑시다. ***

<8-1강_Embedded SQL> 11/11

= SQL을 C나 C++에서 사용하고 싶습니다.

1. static approaches

= c 코드에 SQL을 내장 시킵니다.

- 단점 : 질의가 미리 결정되기 때문에 새로운 질의를 추가하려면 코드 수정이 필요합니다.

ex) Embedded SQL

<Embedded SQL>

- C언어 환경에 SQL을 내장시킵니다.(embedded)

ex) 'EXEC SQL'을 C코드에 작성하고 컴파일을 하면 c-compiler는 ERROR를 발생시킵니다. 그러므로 반드시 **Pre-process(전처리)과정**으로 SQL 문을 약속된 c코드로 변환 시키고 컴파일 합니다.

<Cursors>

- c로 작성된 Application이 DBMS로부터 결과 table을 얻어오면 Application은 자료구조 형태로 저장을 하지 못합니다. Table 형태로 받아온 결과는 저장하지 못하므로 Impedance mismatch(불일치)가 발생합니다.

- 그러므로 Cursor를 사용하여 application이 DBMS로 부터 table을 **tuple 단위로 한 줄씩 읽습니다.**

- Update도 가능합니다.

- Fetch를 사용하면 application에서 선언한 변수에 cursor를 통해 읽어온 tuple에서 특정 속성을 저장 합니다.

EXEC SQL declare myCursor cursor for

select sID, name from student where totalCredit > :creditAmount;

EXEC SQL open myCursor;

EXEC SQL fetch myCursor into :si, :sn;

- 커서를 선언하고 , tuple을 한 줄 씩 읽습니다.
- totalCredit > creditAmount인 tuple에서 sID 와 name을 select 합니다.
- si , sn 변수에 tuple의 속성인 sID , name을 저장합니다.

2. Dynamic approaches

= 사용자가 제공한 query를 application이 동적으로 받습니다.

= SQL API를 사용합니다.

- Static Approaches 보다 많이 사용합니다.

- SQL을 call해서 사용합니다.

- 동적으로 실행되므로 flexible 하지만 느립니다.

ex) ODBC , JDBC

```
stcopy("select name from professor where salary > ?",  
myText); /* can have placeholders */  
EXEC SQL prepare my1 from :myText;  
mySalary = 1000;  
EXEC SQL execute my1 using :mySalary;  
stcopy("select name from professor where pID='234',  
myText2);  
EXEC SQL execute immediate from myText2;  
/* cannot have any placeholders */
```

- 사용자로부터 지속적으로 query를 입력받아서
동적으로 실행합니다.

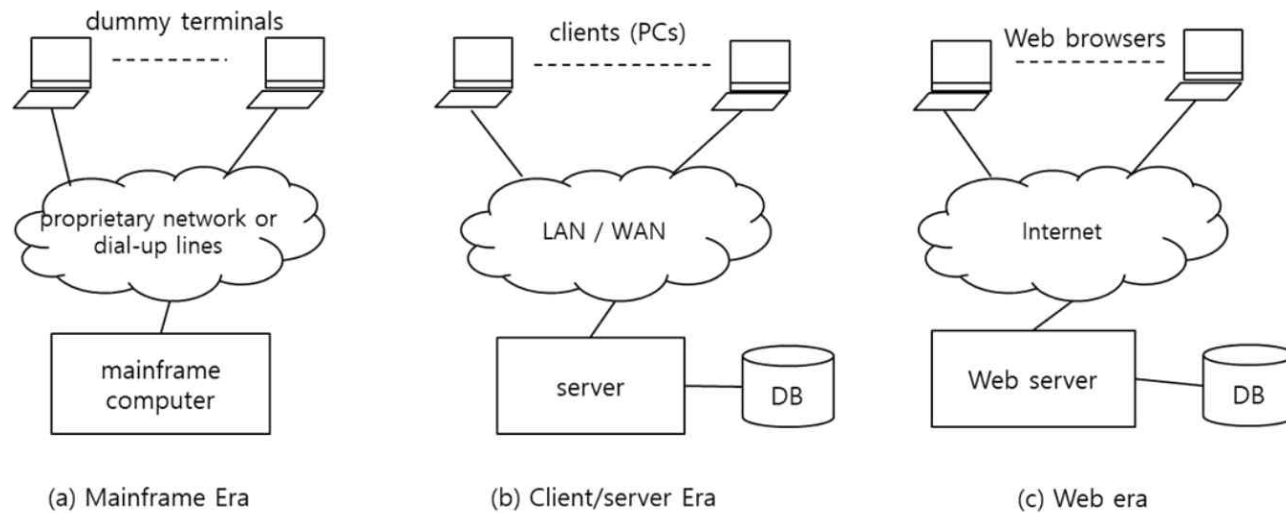
<8-2강_ODBC , JDBC> 11/11

*** 교수님은 스킵 하셨습니다. ***

- 교수님 : API 관련 내용이므로 해당 내용이 추후에 필요하게 되면 따로 찾아서 공부하세요.
- 시험 끝나면 삭제!!!

<8-3강_Application Architecture> 11/11

- 대부분의 사용자는 query를 직접 작성해서 DBMS와 대화하지 않고 application을 사용합니다.
- Database는 사용자의 컴퓨터에 저장되지 않고 **서버에 저장**됩니다.

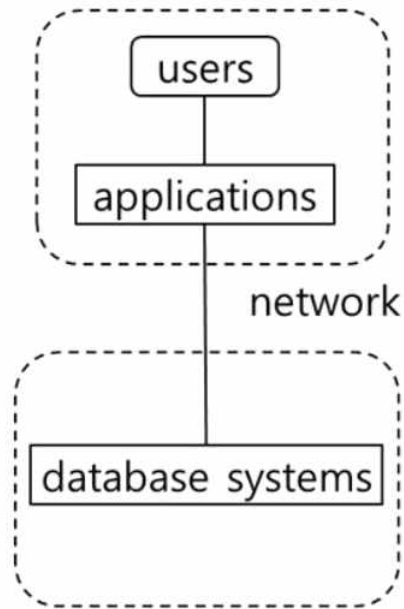


1. Mainframe Era(1960s ~ 1970s)

- Mainframe computer(과거의 성능이 좋은 컴퓨터)와 dummy terminals(문자입력만 되는 사용자의 컴퓨터) 구조였습니다.
- 사용자는 dummy terminals에 명령어를 입력하고 결과만 받아옵니다.

2. Client/Server Era(1980s ~ 1990s)

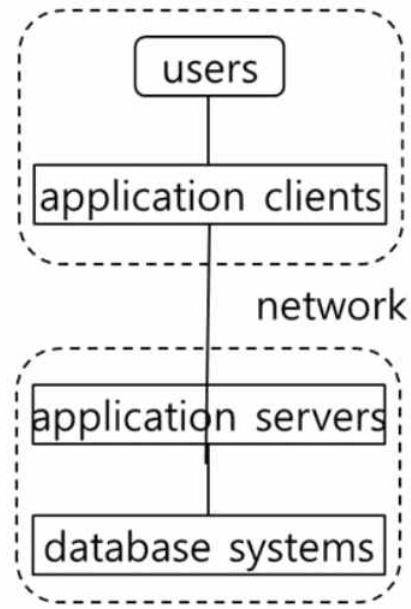
- = Client가 이제 computing power가 있는 적당한 컴퓨터가 되었습니다.
- Client가 서버의 일부기능을 수행할 수 있습니다.



Two-tier architecture

client

server



Three-tier architecture

① 2-tier architecture

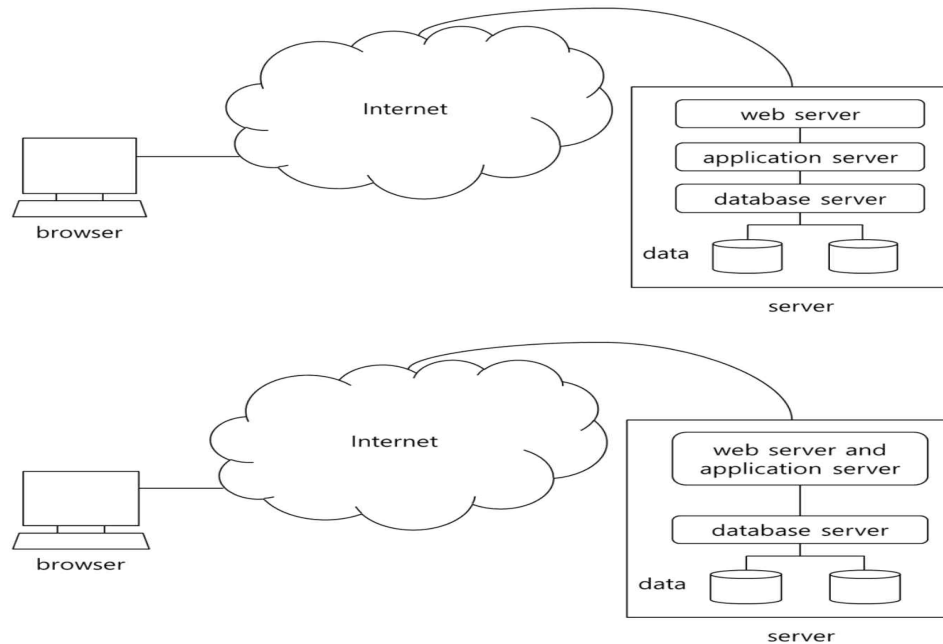
- = Applications를 client에서 사용합니다.
- Application code를 수정하려면 모든 client의 application을 수정해야 하므로 비용이 많이 듭니다.

② 3-tier architecture

- = Server에 Application Servers를 구축해서 server의 code만을 수정하면 됩니다.
- 2-tier 보다 좋은 모델

3. Web Era(1990s ~ 현재)

= Web을 통해 server에서 데이터를 받아옵니다.



① 3-Layer Web Architecture

= Web server , Application server , database server로 server를 3단계로 분할했습니다.

- Web server와 application server간에 교환이 잦기 때문에 과부하가 걸립니다.

② 2-Layer Web Architecture

= 과부하를 막기 위해 web server와 application server를 합쳤습니다.

- 대부분 2-Layer Model을 사용합니다.

<Cookies>

= Text입니다.

- HTTP protocol(web환경)은 connection-less입니다. 그러나 DB는 언제나 connect 되어야 하는 경우가 많습니다.

- 서버에 request를 보낼 때 cookies를 포함해서 보냅니다. Connection-less이므로 connection을 자주 맺어야 합니다. 그 때 마다 사용자 인증을 하는 것이 비용을 많이 발생시키므로 서버에 request를 보낼 때 cookies(사용자 인증 관련 내용)를 포함해서 보냅니다.

- Cookies 덕분에 더 이상 과도한 사용자 인증을 할 필요가 없어졌습니다.

<1번 과제 공부> 11/05

*** 생활코딩 강의를 수강하였습니다. ***

*** 어떤 검색어 뒤에 cheatSheet을 붙이면 잘 정리된 문서를 볼 수 있습니다.***

*** SQL은 대소문자 구분을 하지 않습니다. ***

1. 설치

= 설치를 미리 하였고 , 교수님의 설치 명세서와 일치했습니다.

2. SQL 실행 방법

1) cmd 실행

2) mysql -u root -p

3) 비밀번호 : 1234 입력

<서비스 설정 변경>

① 그림_1

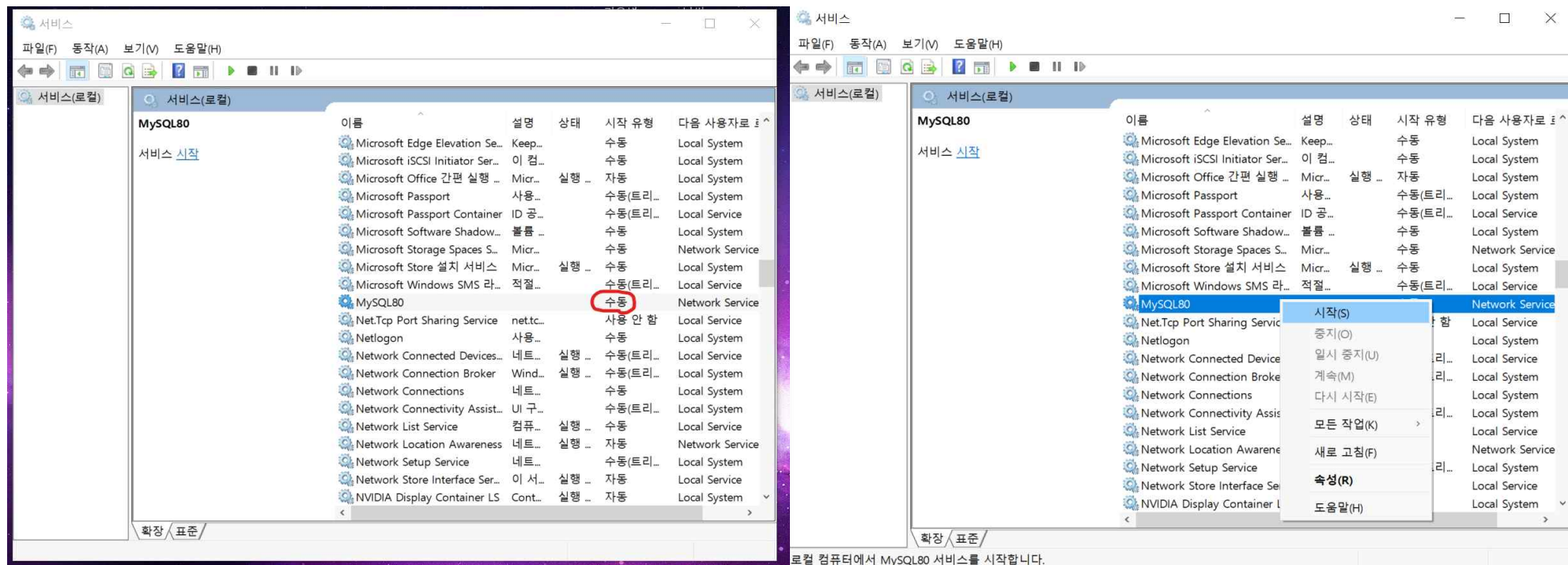
= 평소에 MySQL이 자동 실행되는 것이 좋지 않아 서비스에서 '수동' 모드로 하여 자동으로 실행되지 않게 하였습니다.

- 지금까지는 '자동'으로 설정하여 MySQL이 컴퓨터를 켜면 자동으로 실행되어 cmd에서 MySQL을 사용하는데 문제가 없었습니다. 하지만 이제는 수동이기 때문에 MySQL이 자동으로 실행되지 않아서 MySQL을 실행하면 ERROR가 발생합니다.

② 그림_2

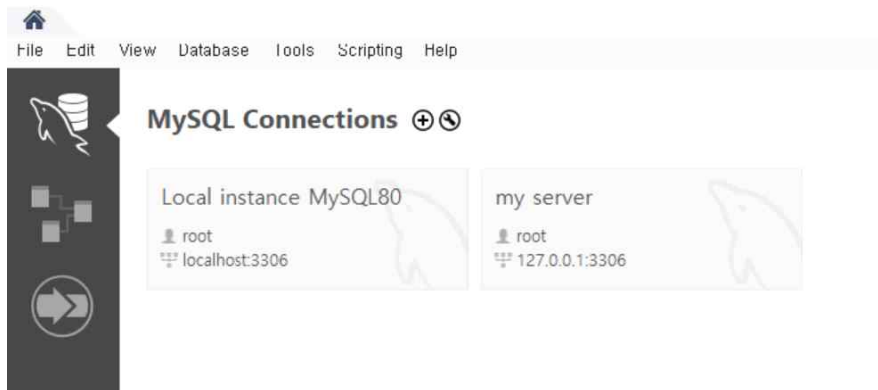
- MySQL을 사용하고 싶다면 수동으로 실행시켜야 합니다.

- 서비스에서 MySQL80을 시작 시킵니다.



3. 기초개념

- DB는 수많은 Table들로 이루어져 있습니다.
- DBMS가 MySQL에서는 MySQL Server 입니다.
- Workbench는 GUI 환경을 통해 사용자가 쉽게 MySQL 명령어를 작성할 수 있게 도와줍니다. 하지만 당분간은 cmd를 통해 직접 명령어를 작성해서 공부합니다.
- Cmd에서 MySQL에 진입했을 때 '->'은 아직 명령어가 다 끝나지 않았으므로 더 진행하라는 의미입니다.
- 127.0.0.1은 workbench가 설치된 컴퓨터의 mysql_server로 접속함을 의미하는 주소입니다.



- VARCHAR(30) = 가변 길이 문자열이며 , 해당 문자열의 최대 크기는 30입니다. 최대 크기를 지정하지 않으면 ERROR를 발생시킵니다.
- 스키마를 만들 때 제약조건을 제대로 작성합니다. 추후에 ALTER를 통해 제약조건을 추가하거나 삭제시킬 수 있긴 합니다.

4. CREATE

1) CREATE DATABASE DB20162471;

= DB20162471인 스키마를 만듭니다.

- 스키마를 만드는 것이므로 테이블들의 집합(데이터의 집합)인 DB를 만든다고 생각해도 무방합니다.
- 테이블 $\subset$ DB

2) CREATE TABLE table이름

= 테이블을 만들 때 제약조건을 설정할 수 있습니다.

1. DESC Table이름;

= 해당 테이블의 스키마를 볼 수 있습니다.

```
mysql> DESC student;
```

Field	Type	Null	Key	Default	Extra
student_id	varchar(45)	NO	PRI	NULL	
student_name	varchar(45)	YES		NULL	
dept_name	varchar(45)	YES		NULL	
admission_date	date	YES		NULL	
home_address	varchar(45)	YES		NULL	

5 rows in set (0.00 sec)

2. SELECT * FROM Table이름;

= 해당 테이블의 모든 tuple을 볼 수 있습니다.

3. USE DB이름; AND SHOW tables;

= 해당 DB에 존재하는 모든 테이블을 볼 수 있습니다.

```
mysql> USE DB20162471;
Database changed
mysql> SHOW tables;
```

Tables_in_db20162471
student

1 row in set (0.00 sec)

4. 제약조건을 새로 만드려는데 이미 제약조건을 무시하는 tuple이 table내부의 있을 때 발생하는 메시지

```
mysql> ALTER TABLE student ADD CONSTRAINT CONST_HOME_ADDRESS CHECK(home_address in('서울' , '부산' , '대구' , '대전'));  
ERROR 3819 (HY000): Check constraint 'CONST_HOME_ADDRESS' is violated.
```

5. 모든 튜플 지우기

6. INSERT

```
mysql> insert into student (student_id , student_name , dept_name , admission_date , home_address) values ('20160001' , '김이변' , '컴퓨터학부' , '2016.03.01' , '대전');  
Query OK, 1 row affected (0.01 sec)
```

7. delete

```
mysql> delete from student where student_id = '20160001';  
Query OK, 1 row affected (0.01 sec)
```

8. alter로 제약 조건 추가

```
mysql> alter table student add constraint address_4 check(home_address in('서울' , '대전' , '대구' , '부산'));  
Query OK, 8 rows affected (0.11 sec)  
Records: 8 Duplicates: 0 Warnings: 0
```

```
mysql> insert into student (student_id , student_name , dept_name , admission_date , home_address) values ('20160001' , '김이변' , '컴퓨터학부' , '2016.03.01' , '대');  
ERROR 3819 (HY000): Check constraint 'address_4' is violated.
```

제약 조건에 맞지 않으면 에러 발생 -> 에러 때 어떤 제약조건인지 나오므로 제약조건명을 잘 씁시다.

9. left(속성,4)

= 속성의 왼쪽에서 4개의 문자열

10. system cls:치면 깨끗해짐.

11.

2. ALTER TABLE 테이블이름

MODIFY COLUMN [*CONSTRAINT* 제약조건이름] *PRIMARY KEY* (필드이름)

12. create table

```
mysql> create table student
-> (
-> student_id varchar(255),
-> student_name varchar(255),
-> dept_name char(255),
-> admission_date date,
-> home_address char(6),
-> primary key (student_id),
-> foreign key (dept_name) references department(dept_name)
-> );
Query OK, 0 rows affected (0.09 sec)
```

13. 속성 추가

```
mysql> alter table takes add grade char(2);
Query OK, 0 rows affected (0.05 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

14. dump 파일 만들기

= mysql 명령어가 아니라 cmd 명령어입니다.

= 관리자 권한으로 해야합니다.

```
C:\WINDOWS\system32>mysqldump -u root -p db20162471 > 20162471_박지원_dump.sql  
Enter password: ****
```

<9-1강_SQL Procedural Extensions> 11/13

<교수님께서 9강에 대해서 정리하신 말씀>

- 함수/프로세스를 SQL에서 지원한다는 정도만 알고 있습니다. 아주 깊은 내용까지는 알 필요 없습니다.
- 함수/프로세스를 SQL에서 왜 지원하는지와, 지원 해줘서 생기는 장/단점은 알아야 합니다.

1. Overview

- SQL:1999에서 정의했습니다.
- PSM(Persistent Stored Modules) : SQL의 절차적인 확장이며, **함수나 프로시저를 미리 정의해서 DBMS 내부에 저장 할 수 있게 되었습니다.**

2. DBMS 내부에서 함수/프로시저를 사용하면 생기는 장점(이러한 장점 때문에 SQL에서 지원합니다.)

1) Encapsulating business logic

= 함수/프로시저 바디 안에 복잡한 로직을 넣어서 다양한 표현을 할 수 있지만 사용자는 내부구조(internal details)를 알지 못하고 사용해도 무방합니다.

2) Avoid network traffic

= 함수/프로시저 Body를 적은 복잡한 query문이 DBMS에 있기 때문에 바로바로 실행이 가능하므로 Network Traffic이 발생하지 않습니다.

3) Delegating access-rights

= Table에 대한 권한이 없어도, 함수/프로시저에 대한 권한이 있으면 접근할 수 있습니다.

4) Possible protection from SQL injection attacks

5) 이미지나 geometric objects 같은 특수한 데이터 자료형을 처리할 수 있습니다.

ex) Polygon Overlap, 두 이미지의 유사도 검사

3. DBMS 내부에서 함수/프로시저를 사용하면 생기는 단점

- 1) Object-code가 DBMS 내부에서 수행될 것이고 corruption(오염, 변질)이 발생하여 DBMS가 멈추는 문제가 발생합니다.
- 2) 허가 되지 않은 데이터에 접근할 수 있으므로 보안에 문제가 발생합니다.

4. 보안문제 해결방법

1) Sandbox

- = Sandbox 안에서만 기능을 사용할 수 있도록 합니다.
- JAVA, C#에서 제공합니다.

2) Object code를 수행하기 위해서 DBMS의 다른 주소에서 프로세스를 새로 형성해서 실행시킵니다.

- 두 개의 프로세스가 생기므로 pipe등을 사용해서 통신합니다.

5. 함수/프로시저 사용 예시

1) 프로시저

```
Create procedure deptCountProc
    (in deptName varchar(20), out count integer)
language C
external name '/usr/shlee/bin/deptCountProc';
```

- deptCountProc : 프로시저 이름
- in deptName : Input
- out count : output
- language C : 프로시저 바디가 C로 만들어 졌습니다.
- '/usr ~ /deptCountProc' : 프로시저 바디가 해당 경로에 Object Code로 존재합니다. 나중에 이 프로시저를 call 하면 해당 경로에 가서 Object Code를 찾아서 프로시저를 처리합니다.

2) 함수

```
Create function deptCountProc2(deptName varchar(20))  
returns integer  
language C  
external name '/usr/shlee/bin/deptCount';
```

- Input에 in을 쓰지 않고 Output에 out 대신에 return을 씁니다.
- 그 외에 다른 내용은 프로시저와 동일합니다.

<9-2강_Functions and Procedures from SQL:1999> 11/13

1. Function

1) 함수 만들기

```
Create function profC(deptName varchar(20)) returns integer
begin
    Declare pCount integer;
    Select count(*) into pCount
    from professor
    where professor.deptName = profC.deptName;
    Return pCount;
end;
```

- 학과를 Input으로 주면 해당 학과에 존재하는 교수님의 수를 Return 하는 함수
- 함수 이름 : profC
- Input(Parameter) : deptName
- Output(Return) : integer 값(pCount)
- 변수 선언 : Declare pCount integer
- Where 절(조건) : **profC.deptName(=함수의 Input)** 일치하는 professor_table의 deptName
- 이 명령어를 수행하면 DB에 stored function으로 저장됩니다.

2) 함수 사용

```
Select deptName, budget  
from department  
where profC(deptName) > 5;
```

= profC를 call해서 주어진 학과의 교수님의 수가 5보다 큰 학과에서 deptName과 budget을 찾습니다.

3) Return 형이 Table인 함수

```
Create function myProf(deptName varchar(20))  
returns table(  
    pID char(5),  
    name varchar(20),  
    deptName varchar(20),  
    salary numeric(10,2))  
return table(  
    select pID, name, deptName, salary  
    from professor  
    where professor.deptName = myProf.deptName);
```

Select * from table(myProf('CS'));

- Table을 return할 때 속성에 대한 스키마를 명시하고 , 어떤 조건의 속성을 갖는 tuple을 선택해서 table을 만들지도 명시합니다.
- Return 값이 table이므로 from절에서 table을 붙여서 사용할 수 있습니다.
- SQL:2003부터 지원합니다.

2. 프로시저

```
Create procedure profC2 (in deptName varchar(20), out pCount integer)
begin
  Select count(*) into pCount from professor
  where professor.deptName = profC2.deptName;
end
```

```
Declare myProcCount integer;
Call profC2('CS', myProcCount);
```

- = 함수는 여러 개의 output을 나타내지 못하므로 , 이런 경우에 프로시저를 사용합니다.
- 원리는 함수와 비슷합니다.

3. Procedural Constructs

- loop(while , for , iteration), if문 , exception을 지원합니다.

*** 9-3강 PL/SQL은 스킵 하셨습니다. ***

<과제 2번 공부> 11/24 ~ 11/26

- 기본적으로 SQL에서 ()는 순서를 조절합니다. 그러므로 불필요한 ()를 사용하면 안 됩니다.
- From 절에서 카타시안 곱으로 테이블들을 합친다면 겹치는 속성은 중복되어서 결과테이블에 나타납니다. 그러므로 중복된 속성들은 반드시 SQL 문 어디에서든 속성 앞에 '테이블이름 + .'을 붙여줍니다. 겹치지 않는 속성은 붙이지 않아도 무방합니다. 이 과정이 매우 귀찮기 때문에 대부분 natural join을 사용하였습니다.
- Aggregate Functions는 반드시 단독으로 사용하지 않고 질의문의 결과 테이블로 사용해야 합니다.
- professor as p : professor table을 앞으로 p로 쓰겠다는 의미이므로 굳이 사용하지 않아도 됩니다.

<문제 풀이 : 대부분 3강의 기초 base + 5강의 내용에 관한 문제이고 , 12번과 13번이 6-2강에 관한 문제입니다.>

1. 1번 문제

- 매우 간단한 문제입니다.

2. 2번 문제

- in , not in을 사용했습니다.

3. 3번 문제

- Avg()와 중첩 Query를 사용했습니다.

4. 4번 문제

- 집계함수에 조건을 추가하여 조건에 해당하는 속성의 값만 계산하도록 하였습니다.
- group by를 통해 집계함수를 속성별로 구분하였습니다.

ex) select student_name , sum(case when grade = 'F' then 0 else credit end) from student natural join takes natural join course where student_id in (select student_id from takes where grade = 'F') group by student_name;

5. 5번 문제

- 성이 '김'인 학생 : `student_name like '김%'`
- 4번 문제와 유사합니다.

6. 6번 문제

- 4번 문제와 유사합니다.

7. 7번 문제

- 제대로 해결하지 못했습니다.

8. 8번 문제

- 매우 간단한 문제입니다.

9. 9번 문제

- 매우 많은 table을 natural join 해 본 문제입니다.
- 집계함수(5-1강) + join(5-2강) + sub-query(5-3강)가 모두 포함 된 좋은 문제입니다.
- 수강한 과목이 없는 학생의 경우도 from에 존재하는 table에서 찾아야 하기 때문에 natural join 대신 , natural left join OR natural right를 사용합니다. 다시 말해 , 찾고자 하는 데이터를 위해 고의적으로 table의 크기를 늘려줍니다. 대부분의 속성이 null인 tuple도 많이 발생하겠지만 그 tuple에서 null이 아닌 속성이 중요한 경우도 존재합니다.

10. 10번 문제

- 중첩 query가 매우 긴 문제입니다.

11. 11번 문제

- 재귀적 query 문제입니다.
- 해결하지 못했습니다.

12. 12번 문제

- 6-2 강의(Integrity Constraints)에 관련된 내용을 담고 있습니다.
- 3-3강의 update에 나온 update 순서에 관한 문제입니다.

ex) update professor set salary = case when salary < 70 then salary * 1.05 else salary * 0.97 end;

13. 13번 문제

- 6-2 강의(Integrity Constraints)에 관련된 내용을 담고 있습니다.
- 4년 초과 차이나는 입학년도를 구하는 문제입니다. date , subdate , interval을 적절하게 사용해서 해결했습니다.

ex) delete from student where date(admission_date) < date(subdate('2020-03-01' , interval 365*4 + 1 day));

<10-1강_Entity and Relationship> 11/22

*** 교수님 : 많은 시간을 할애해서 10강을 공부해야 함 ***

1. Attributes(속성)

= Entity OR Relationship이 가지는 특성.

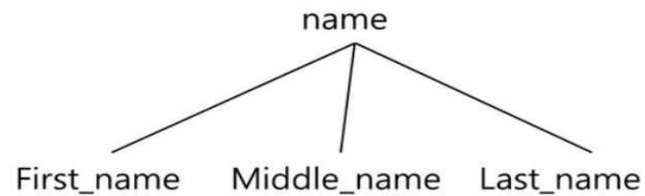
- 관계형 DB에서의 속성과 비슷한 개념입니다.

1) Simple vs Composite

- 간단한 속성 : Simple

- 복잡한 속성 : Composite

<Example : professor(entity) 중에서 composite한 속성인 name(attribute)>



2) Single-valued vs Multi-valued

- 속성의 값이 1개 : Single-valued

- 속성의 값이 여러 개 : Multi-valued

3) Derived Attributes

= 유추 가능한 속성

ex) 학생의 나이와 생년월일 속성이 있다고 할 때 , 생년월일을 통해 나이를 유추 할 수 있습니다.

2. Entity(객체)

= 속성(attribute)들의 집합

- Entity는 자신이 가지고 있는 원소를 나타낼 수 있는 속성(Attribute)을 가지게 됩니다.
- 관계형 DB의 table과 유사한 개념입니다.

1) Entity vs Entity Set

- Entity들을 모아서 만든 집합을 Entity Set이라고 합니다.
- Entity가 정글챔피언이면 Entity Set은 entity(정글챔피언)의 집합이므로 , 리신 + 엘리스 + 누누 + etc 입니다.

2) Entity vs Instance

- Entity가 정글챔피언이면 리신은 instance고 , 엘리스도 instance고 , 누누도 instance 입니다.

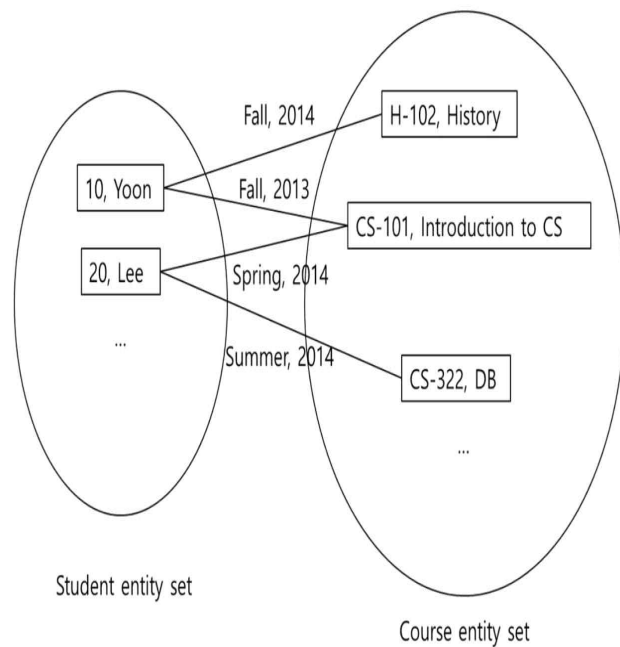
3) Entity vs Attribute

- Entity가 정글챔피언이면 Attribute는 이름 , 공격력 , 주문력 , 이동속도 , etc 입니다.

3. Relationship(관계)

= Entity와 Entity간의 연관성을 표시하는 개념

- Relationship들을 모아서 만든 집합을 Relationship Set이라고 하며 이를 Takes 라고도 부릅니다.

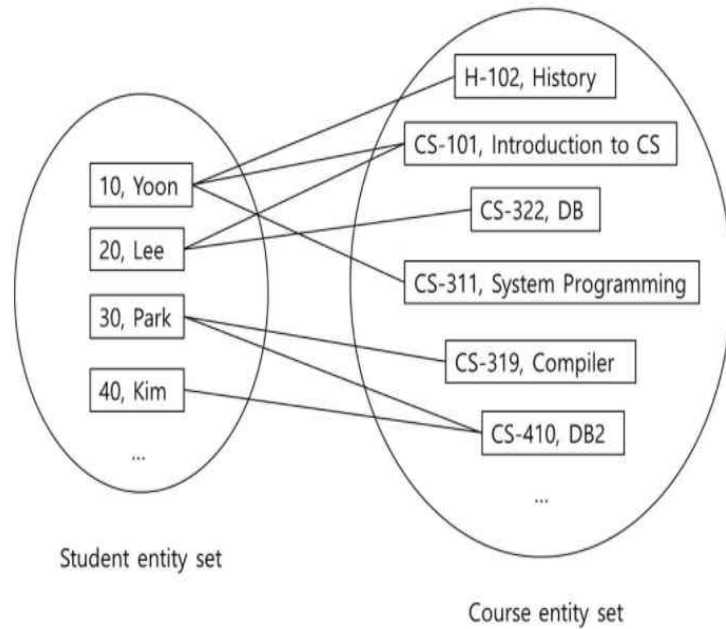


- Relationship에도 속성이 존재합니다.
- 그림에서 'Fall, 2014'이 Relationship의 속성입니다.

<Degree>

- Relationship을 나타낼 때 참여하는 Entity Set의 개수
- 2 : Binary , 3 : Ternary, 4 : Quaternary, 5 : Quinary
- 대부분 Binary 이며 Quaternary 와 Quinary는 이론적인 내용입니다.

1) Entity 와 Relationship 그림



- Student entity set에는 4명의 entity가 존재합니다.
- Course entity set에는 6개의 entity가 존재합니다.
- entity 끼리의 Relationship을 연결선으로 표현합니다.

2) Cardinality Constraints

- 두 개의 Entity-Set 구조(Binary Relationship)에서 Entity-Set을 각각 A,B라고 합시다. A에는 a1 , a2 , a3 , etc / B에는 b1 , b2 , b3 , etc 의 entity로 구성되어 있습니다.

① one to one

= A의 entity 1개는 relationship이 존재한다면 반드시 B의 entity 1개와만 대응해야 합니다.

② one to many

= A의 entity 1개는 relationship이 존재한다면 B의 entity 여러 개와 대응할 수 있습니다.

③ many to one

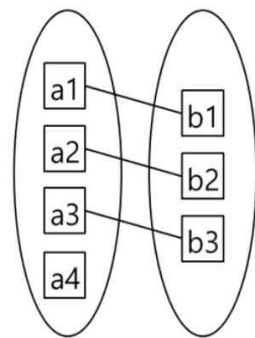
= B의 entity 1개는 relationship이 존재한다면 A의 entity 여러 개와 대응할 수 있습니다.

- one to many의 역 관계입니다.

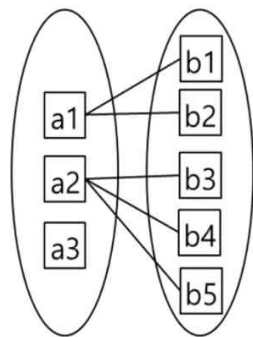
④ many to many

= one to many 와 many to one을 합친 개념입니다.

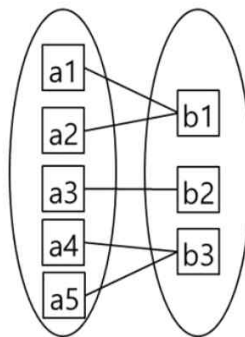
- 네 가지 경우 모두 관계를 맺지 않는 객체가 존재해도 괜찮습니다.



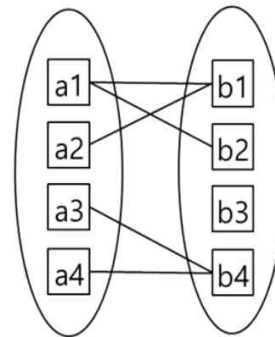
One to one



One to many



Many to one



Many to many

4. Keys

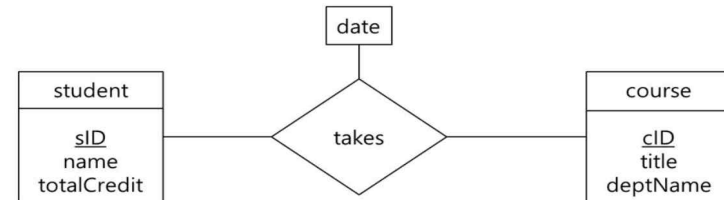
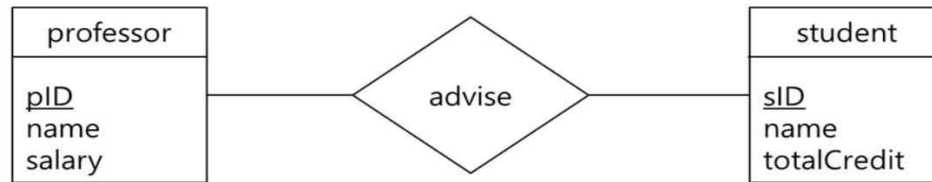
- 관계형 데이터베이스에서 배운 key와 동일한 개념입니다.
- Super key , Candidate key , Primary key
- 관계형 데이터베이스에서는 Key가 속성이었다면 E-R에서는 relationship도 key가 될 수 있습니다.
- Cardinality에 따라 key의 형태가 바뀝니다.

<10-2강_E-R Diagrams> 11/23

*** E-R Diagrams를 해석할 수 있어야 합니다. ***

1. E-R Diagrams 기호

1) 기본 기호



① Entity set : 네모

ex) professor

② Attribute : 네모 안에 적어줍니다.

ex) pID , name , salary

③ Relationship set : 다이아몬드

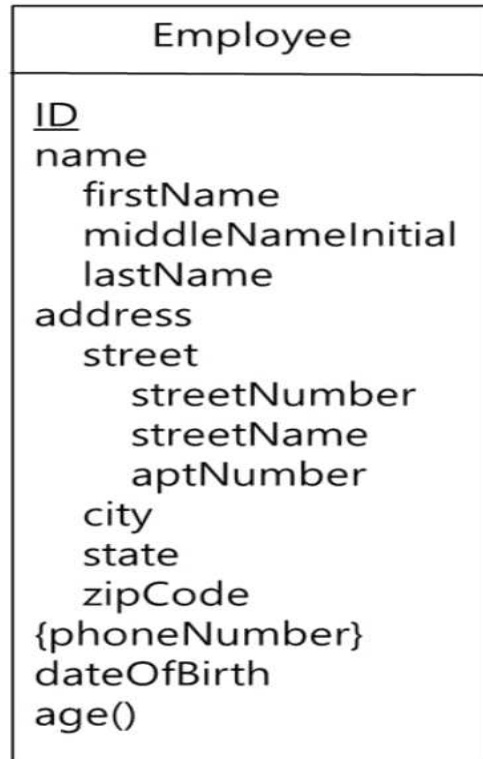
ex) advise

④ Primary key : 밑줄

ex) pID , sID , cID

⑤ Relationship(takes)에 date라는 attribute를 가질 수 있습니다.

2) Attribute 기호



- ① Simple Attribute : ID , dateOfBirth
- ② Composite Attribute : name , address
 - Name(Composite Attribute)을 구성하는 firstName , middleNameInitial , lastName을 각각 component라고 부릅니다.
 - Address는 내부에 또 하나의 Composite Attribute인 street이 존재합니다.
- ③ Single-valued Attribute : phoneNumber 제외
- ④ Multi-valued Attribute : phoneNumber
 - Multi-valued Attribute는 set 기호인 { }를 사용합니다.
- ⑤ Derived Attribute : age()는 dateOfBirth를 보고 유추할 수 있으므로 함수형으로 나타냅니다.

3) Cardinality Constraints 기호

<Directed line>

= 화살표가 있고 one에 해당합니다.

<Undirected line>

= 화살표가 없고 many에 해당합니다.

① one to one



② many to one



③ many to many



- many to one을 해석하면 학생 하나는 다수의 교수님과 관계를 맺을 수 있습니다.

4) Participation Constraints 기호

① Total Participation

= Entity set의 모든 entity가 relationship에 참여합니다.

- Double line으로 표현합니다.

ex) professor와 student 관계에서 모든 학생은 관계에 참여합니다.

② Partial Participation

= Entity set의 일부 entity만이 relationship에 참여합니다.

- Single line으로 표현합니다.

ex) professor와 student 관계에서 일부 교수님만 관계에 참여합니다.

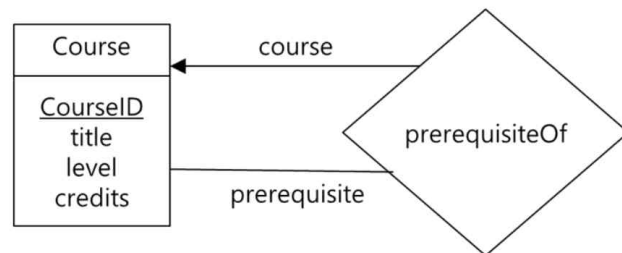
5) Roles

- 참여하는 entity set이 동일한 경우에 사용합니다.

<Example : Binary Relationship 구조에서 Entity set이 Course로 동일한 경우>

- Pre-requisiteof는 선수과목을 의미하므로 필요한 Entity set이 2개의 course입니다.

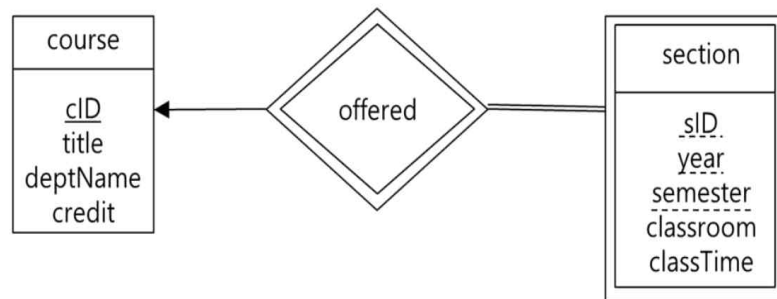
- Pre-requisiteof를 recursive relationship이라고 부릅니다.



6) Weak Entity Set

- = 모든 Entity set은 당연히 primary key가 존재해야 합니다. 하지만 자체적으로 primary key가 존재하지 않아 강제적으로 primary key를 만들어 주어야 하는 Entity Set.
- Weak entity set은 자체적으로 primary key는 존재하지 않지만 자체적으로 partial key(=discriminator)는 존재합니다. Entity set 전체에서 식별은 불가능하지만 entity set의 부분집합인 그룹에서는 식별이 가능한 key입니다.
- Weak entity set의 primary key = Strong entity set의 primary key + Weak entity set의 partial key. 다시 말해 , weak entity set의 attribute에 strong entity set의 primary key가 존재해야 합니다.
- 대부분 primary key가 존재하는 strong entity set 이지만 가끔 weak entity set이 존재합니다.
- Weak entity set은 Double 네모로 표현합니다.
- 반드시 Strong entity set과 relationship을 맺어야 사용이 가능합니다. 이 관계를 double diamond로 표현합니다.
- Strong entity set과 weak entity set은 반드시 one to many 관계이며 , Strong이 one이고 Weak이 many입니다.
- Weak entity set은 반드시 total participation입니다.

<Example>



- Course의 entity : DB , OS
- Section의 entity : (가,2020,fall), (나,2020,fall), (가,2020,fall), (나,2020,fall)
- Section의 entity는 겹치기 때문에 primary key가 없습니다. 그러므로 앞에 entity 2개를 묶고 , 뒤에 entity 2개를 묶어서 그룹화합니다.
- Partial key : 점선
- Weak Entity Set : Section
- Strong Entity Set : Course
- Course가 strong이므로 one이고 , Section이 weak이므로 many.
- Course의 entity는 여러 개의 section의 entity와 관계를 가질 수 있습니다.

<10-3강_E-R Diagram -> Table> 11/24

1. Overview

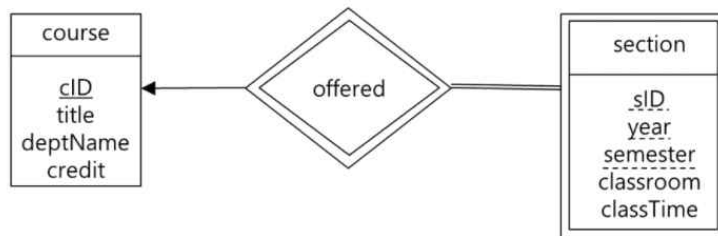
- 이번 강의의 주제는 E-R Diagram을 어떻게 table로 변환시키는지에 대한 강의입니다.
- Table로 변화시키려면 **primary key**가 필요합니다.
- **Entity set**은 table이 됩니다.
- **Relationship set**은 table이 됩니다.
- **Attribute**는 table의 속성(column)이 됩니다.

2. 변환 방법 분류

1) Weak Entity Set

= one to many 관계이므로 relationship에 해당하는 table을 만들지 않습니다.

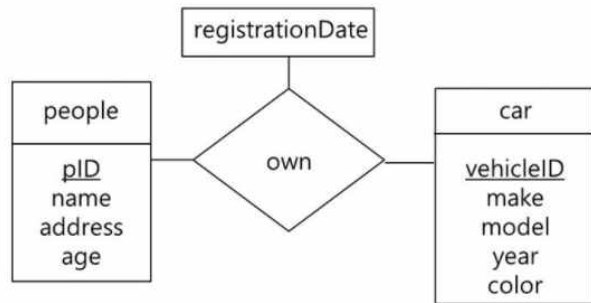
- course(cID, title, deptName, credit)
- section(cID, sID, year, semester, classroom, classTime)
- No table for "offered"



- Weak Entity Set에는 strong entity set의 primary key를 포함시켜야 합니다.
- Relationship Set에 해당 하는 offered는 테이블을 만들 필요가 없습니다.
- 만들어 봤자 중복이 생깁니다.

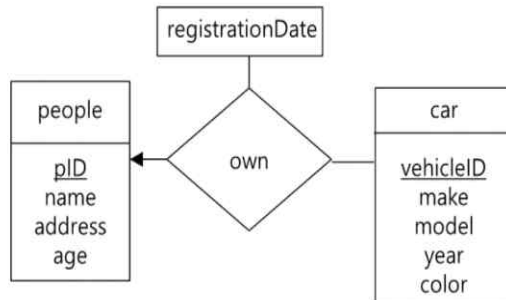
2) Many to Many

= 2개의 entity set을 table로 만들고 , relationship의 attribute에 entity set들의 primary key를 추가하여 table로 만듭니다.



- People 과 Car에 대해서는 Table을 만듭니다.
- People 과 Car의 primary key를 포함시켜서 relationship set을 Own(pID , vehicleID , registrationDate)형태의 table로 만듭니다.
- Primary Key는 pID + vehicleID가 됩니다.

3) One to Many & Many to One



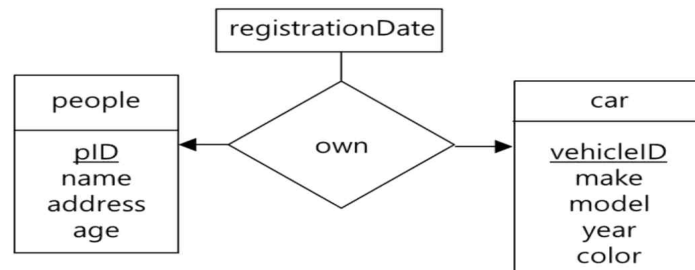
- people(pID, name, address, age)
own(pID, vehicleID, registrationDate)
car(vehicleID, make, model, year, color)
- people(pID, name, address, age)
car(vehicleID, make, model, year, color, registrationDate, pID)

① 2개의 entity set을 table로 만들고 , relationship의 attribute에 entity set들의 primary key를 추가하여 table로 만듭니다.

② Relationship_table을 만들지 않고 2개의 entity set_table로만 변환이 가능합니다. 단 , entity set(one)의 primary key와 relationship set의 attribute를 entity set(many)의 attribute로 추가를 해야 합니다. (Weak Entity Set과 동일한 방법)

4) one to one

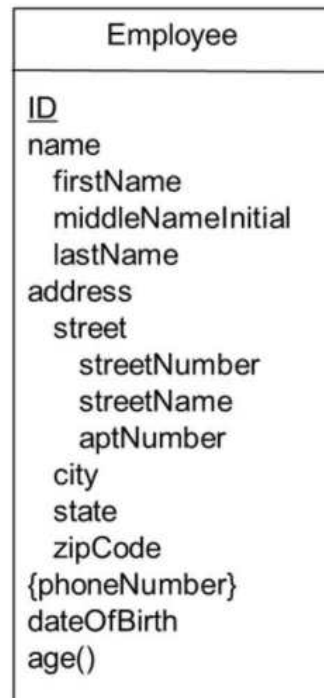
= Entity set 하나를 many로 생각해서 one to many & many to one과 완전히 동일한 방법으로 변환 합니다.



- people(pID, name, address, age)
own(pID, vehicleID, registrationDate)
car(vehicleID, make, model, year, color)
- people(pID, name, address, age)
car(vehicleID, make, model, year, color, registrationDate, pID)
- people(pID, name, address, age, registrationDate, vehicleID)
car(vehicleID, make, model, year, color)

5) 다양한 형태의 attribute

- ① Composite attribute : Composite attribute는 table의 속성으로 선언 하지 않고 각각의 component를 모두 table의 속성으로 선언 합니다.
- ② Derived attribute : table의 속성으로 선언합니다.
- ③ Multi-valued attribute : 관계형 데이터 모델에서 set-type를 지원하지 않으므로 table의 속성으로 선언 할 수 없습니다.



Employee(ID, firstName, middleInitial,
lastName, streetNumber, streetName,
aptNumber, city, state, zipCode,
dateOfBirth, age)

<10-4강_Design Issues> 12/04

- 사람마다 실세계의 데이터를 디자인하는 방식이 너무나도 다릅니다.
- 이 강의에서는 뭐가 좋고 나쁨을 판단하는 것이 아니라, 다양한 표현방법을 배웁니다.
- 필기에서는 entity set과 entity를 구분하지 않고 적겠습니다. (교수님 강의에서도 그렇게 하십니다.)

An attribute vs. an entity set

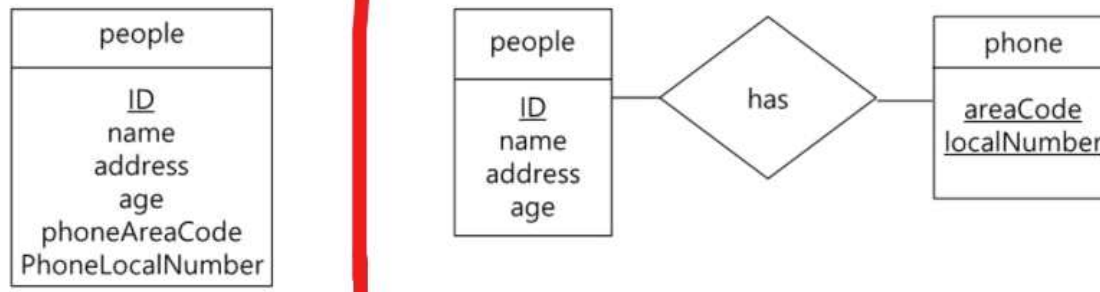
An entity set vs. a relationship set

A ternary relationship vs. a pair of binary relationships

Strong vs. weak entity set

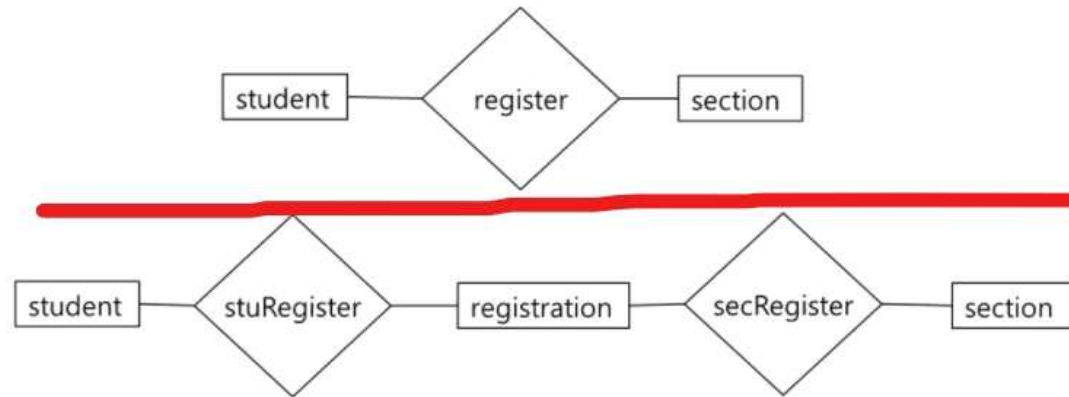
- 어떤 데이터를 attribute로 표현하기 vs 어떤 데이터를 entity set으로 표현하기
- 아래 3개도 비슷한 맥락으로 이해합니다.

1. Attributes vs Entity Set



- 사람은 사람에 대한 정보 + 핸드폰 번호에 대한 정보를 갖고 있습니다.
- 왼쪽 그림은 한 객체에 모두 속성으로 저장하고, 오른쪽 그림은 두 객체의 속성으로 저장하고 관계를 나타내고 있습니다.
- 핸드폰 관련 속성을 추가할 때는 오른쪽 그림 구조가 더 좋습니다.

2. Entity Set vs Relationship Set

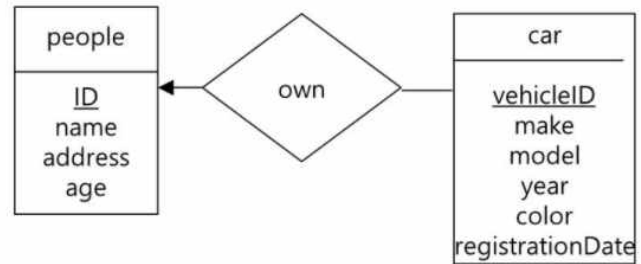


- 대학에서 학생과 강좌에 대한 관계입니다.
- 위의 그림은 등록을 관계로 표현하고, 아래 그림은 등록도 객체 집합으로 만들어 버립니다.
- Action에 관한 것은 관계로 표현하는 것이 좀 더 일반적입니다.

3. 관계가 속성을 갖는 경우

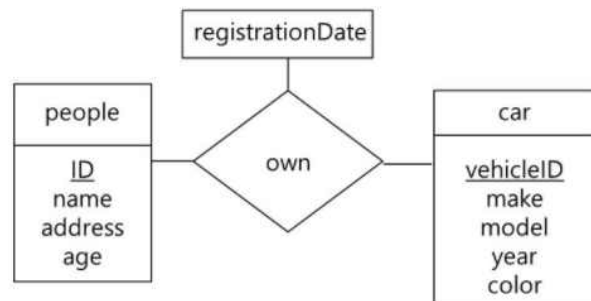
- one to one, many to one, many to many에 따라서 어떤 경우는 people 이나 car 객체 집합에 넣을 수도 있습니다.

1) One to Many



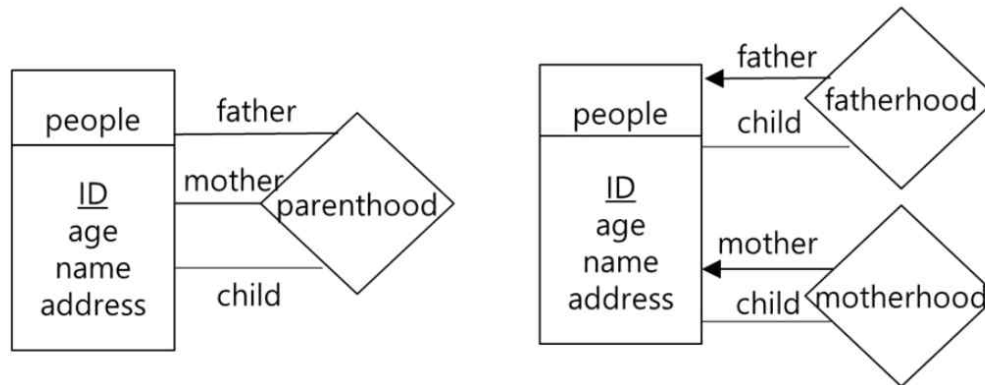
- car 객체에 넣을 수 있습니다.

2) Many to Many



- people 이나 car 객체 집합에 넣을 수 없습니다.

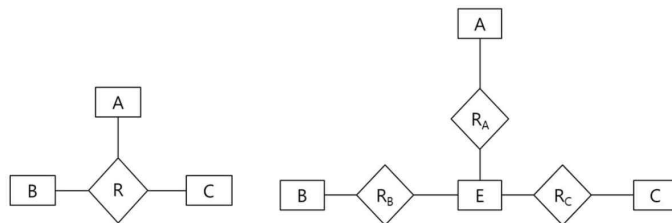
4. Binary vs Non-binary Relationships



- 왼쪽 그림은 3개의 entity로 구성된 ternary relationship이고, 오른쪽 그림은 2개의 entity로 구성된 binary relationship을 2개를 만들었습니다.
- Ternary Relationship을 만들기 위해서는 반드시 3개의 entity가 필요하기 때문에 3개의 entity를 만족하지 못하면 표현할 수 없습니다.
- 2개의 Binary Relationship은 2개의 entity만 만족하면 되므로 Ternary Relationship 보다 적은 entity로도 표현할 수 있습니다.

<복잡한 Relationship을 다수의 Binary Relationship으로 변환>

= 교수님께서 중요하지 않다고 하셨습니다.



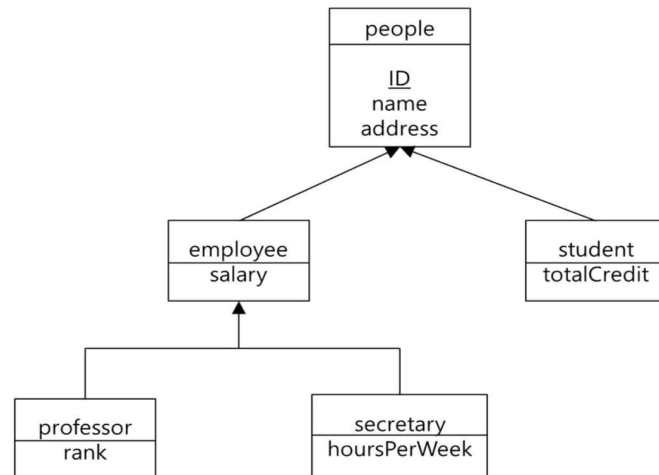
- Ternary(R) -> Binary(R_A + R_B + R_C)

<10-5강_Extended ER Model> 12/15

1. Specialization / Generalization Overview

<Example : 대학교 DB>

*** 이 예제는 10-5강에서 계속 사용합니다. ***



- Specialization : DB를 Top-Down 방식으로 해석
- Generalization : DB를 Bottom-Up 방식으로 해석
- people : 대학에 존재하는 사람
- employee : 직원
- student : 학생
- professor : 교수
- secretary : 비서

<상속>

- employee는 people의 모든 속성을 상속받아서 4개의 속성(people의 속성 3개 + salary)을 갖습니다. student도 마찬가지입니다.
- professor는 employee의 모든 속성을 상속받아서 5개의 속성(employee의 속성 4개 + rank)을 갖습니다. secretary도 마찬가지입니다.

2. Specialization / Generalization의 제약

1) Condition-defined : 부모 Entity Set의 관점에서 **자식 Entity Set은 조건에 따라 분류**됩니다.

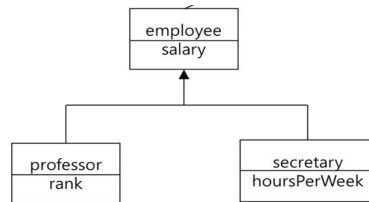
ex) People에서 돈을 받는 사람은 employee 이고 돈을 내는 사람은 student라고 조건을 추가해서 분류할 수 있습니다.

2) User-defined : employee를 정규직 , 비정규직으로 나뉘었을 때 비정규직에서 **몇 가지 조건을 만족**시키면 정규직으로 **변경**됩니다.

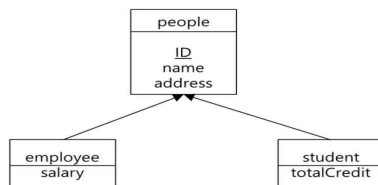
3) Disjoint vs Overlapping

- Disjoint와 Overlapping은 반대의 개념입니다.

① Disjoint : employee를 professor , secretary로 나뉘었을 때 반드시 professor나 secretary 중 **하나에만 포함**되어야합니다.



② Overlapping : 박사과정인 학생은 employee 면서 **동시에** student입니다.



4) Total vs Partial

- People(entity set)에 속하는 모든 Entity의 집합을 {P1 ~ Pn}이라고 합니다.

① Total : P1 ~ Pn이 employee OR student에 **전부 빠짐없이 참여**하는 경우

② Partial : P1 ~ Pn이 employee OR student에 **일부분만 참여**하는 경우

3. Specialization된 E-R Diagram을 관계형 데이터베이스로 표현하기

1) 방법_1 : Join

schema	attributes
people	ID, name, address
student	ID, totalCredit
employee	ID, salary

- 부모의 속성은 자식에 적지 않고 , foreign key(ID)만 적어줍니다.
- People , Student , Employee는 table로 만듭니다.
- Student와 Employee에 대한 정보를 가져오기 위해서는 ID 속성을 통해 People과의 부모-자식 관계를 맺습니다.
- Student와 Employee에 대한 정보를 가져오는 경우 People을 ID를 통해 참조하며 동시에 people의 name 과 address도 알게 됩니다.
- 문제점 : Student OR Employee에 대한 정보를 가져오기 위해 반드시 Join을 해야 합니다.

2) 방법_2 : Join 하지 않기.

schema	attributes
people	ID, name, address
student	ID, name, address, totalCredit
employee	ID, name, address, salary

- 부모의 속성을 모두 자식 Table에 적어주어 join이 필요하지 않도록 합니다.

<문제점>

- ① 모든 사람이 Student 혹은 Employee에 속하므로 total이 되어 People_Table이 필요가 없게 됩니다. 하지만 People_Table을 참조하는 또 다른 Table이 존재하면 무결성 제약을 깨지 않기 위해 People Table이 필요합니다.
- ② 데이터의 중복이 발생합니다.

<11-1강_Good schema and Bad schema> 11/28 ~ 12/02

1. 교수님 서론

- 11강은 방대한 내용이 있고 어렵지만 제가 가르치는 내용만 공부하세요.
- DB 정규화와 관련된 내용이므로 이해하기가 어려울 겁니다.

2. Good schema and Bad schema

- 사람들마다 DB를 설계 하는 방식이 다양합니다. 그러므로 잘 만든 DB(Good)와 잘 만들지 못한 DB(Bad)에 대하여 판단할 줄 알아야 합니다.

1) Bad Schema

- Bad schema는 3가지의 Anomaly(이상)를 갖습니다. : **Update anomaly** , **Delete anomaly** , **Insert anomaly**

<Example_1>

mybadtable1(cID, title, deptName, credit, chairman,
building, budget)

cID	title	deptName	credit	chairman	building	budget
202	Java	CS	2	Lee	IT building	67000
203	Data Structure	CS	3	Lee	IT building	67000
301	Databases	CS	3	Lee	IT building	67000
241	Logic Circuit	EE	2	Han	HN Eng	82500
341	LAN	EE	3	Han	HN Eng	82500
342	Electronics	EE	3	Han	HN Eng	82500
222	Computer Art	Media	3	Paik	IT building	55500
223	Game Theory	Media	3	Paik	IT building	55500

- Course 관련 속성 : cID ~ credit
- Department 관련 속성 : chairman ~ budget
- 현재의 table은 중복되는 데이터가 매우 많기 때문에 좋지 않은 Table입니다.
- **중복이 적을수록 좋은 table**입니다.
- Update Anomaly : 'Lee'학과장이 바뀐다면 관련된 3개의 tuple을 모두 update 해야 합니다.
- Delete Anomaly : CS 학과의 course 관련 속성(정보)을 삭제하면 department에 대한 정보도 사라져 버립니다.
- Insert Anomaly : Course 관련 정보를 추가하기 전에는 department에 대한 정보를 insert할 수 없습니다.

<Example_2>

mybadtable2(clD, title, deptName, credit, roomID,
building, capacity)

- Course와 Room에 대한 속성을 갖는 table입니다.
- 3개의 anomaly가 모두 발생합니다.

2) good schema(Design goal)

- Bad form 이 아닌 good form으로 table을 design 해야 합니다.
- N개의 table 중에서 table_2가 bad schema 라면 table_2를 good schema가 될 때 까지 **decompose(분해)** 합니다.

3. **Functional Dependency** 다음 장의 Example을 정독하면 매우 이해가 쉬울 것 입니다.

어떤 테이블 R 에 존재하는 필드들의 부분집합을 각각 X 와 Y 라고 할 때, X 의 한 값이 Y 에 속한 오직 하나의 값에만 사상될 경우에 " Y 는 X 에 함수 종속 (Y is functionally dependent on X)"이라고 하며, $X \rightarrow Y$ 라고 표기한다.

예를 들어, 테이블에 [생일]과 [나이]라는 필드가 존재할 경우에 [나이] 필드는 [생일] 필드에 함수 종속이다. 즉, 생일을 알고 있다면, 나이에 대한 필드를 참조하지 않거나, 아예 필드를 유지하지 않아도 나이를 결정할 수 있다. 데이터베이스 설계 단계에서 함수 종속 관계에 있는 필드를 찾는다면, 그 만큼 중복된 데이터를 줄일 수 있다. 그러므로, 데이터베이스 설계 단계에서 각 정보들 간의 함수 종속 관계를 찾는 것은 매우 중요하다.

- r : Table
- R : Schema
- 하지만 r 과 R 을 엄밀하게 구분할 필요는 없습니다.
- Instance 개념을 두 가지 관점으로 볼 수 있습니다. 테이블의 관점으로 보면 '10.1'에서 배운 내용으로 Instance를 해석할 수 있고 속성의 관점으로 보면 속성은 여러 개의 값을 가질 수 있으며 이 값들을 Instance로 해석할 수 있습니다. 여기서는 후자로 해석합니다.
- Functional Dependency의 형태를 알파 $\rightarrow$ 베타 로 만들 수 있습니다. 이는 ' $t1[\alpha] = t2[\alpha]$ 라면 $t1[\beta] = t2[\beta]$ '를 의미합니다.
- Functional Dependency의 형태를 아마 다양하게 할 수 있을 겁니다.
- 공부하고 스스로 이해한 개념 : ' $A \rightarrow B$ '의 의미는 속성 A 의 값이 같으면 속성 B 의 값도 언제나 같아야 합니다. 예를 들어 , A 가 핸드폰 명칭이고 B 가 제조회사라면 A 의 instance가 Galaxy A51인 모든 tuple에서 B 의 instance는 삼성으로 모두 같아야 합니다.

<Example>

A	B
1	4
1	4
2	4
3	5

- ① Functional Dependency : $A \rightarrow B$
- 속성 A 의 모든 instance(1,1,2,3)가 속성 B 의 모든 instance(4,4,4,5)에 대해서 $A \rightarrow B$ 를 만족하는지 확인해야 합니다.
 - 속성 A 의 instance가 1이면 속성 B 의 instance도 모두 4 입니다.
 - 속성 A 의 instance가 2 , 3은 중복되지 않기 때문에 $A \rightarrow B$ 를 확인할 필요가 없습니다.
 - Table의 모든 instance에서 $A \rightarrow B$ 가 만족하므로 Functional Dependency $A \rightarrow B$ 는 R 에 대해서 holds on 하다고 합니다.

② Functional Dependency : $B \rightarrow A$

- 속성_B의 모든 instance(4,4,4,5)가 속성_A의 모든 instance(1,1,2,3)에 대해서 $B \rightarrow A$ 를 만족하는지 확인해야 합니다.
- 속성_B의 instance가 4이면 속성_A의 instance가 1 OR 2 입니다. 그러므로 Functional Dependency를 만족하지 않습니다.
- Table의 **모든** instance에서 $B \rightarrow A$ 를 만족해야 하지만 , 만족하지 못하는 경우가 존재하기 때문에 Functional Dependency $B \rightarrow A$ 는 R에 대해서 holds on 하지 않습니다.

4. Key

- Key는 가장 일반적인 Functional Dependency 입니다.
- 아까 예제에서 학과이름이 같으면 $\rightarrow$ chairman , building , budget 모두 다 같아야합니다.

- 알파 , 베타 : 속성 / R : 스키마

1) 어떤 알파가 super key

$$\alpha \rightarrow R$$

2) 어떤 알파가 candidate key

$$\alpha \rightarrow R \text{ and}$$

$$\text{for no } \beta \subset \alpha, \beta \rightarrow R$$

- 1번 조건 : Candidate key는 super key + minimum을 만족해야 하므로 super key의 조건은 당연히 만족해야 합니다.
- 2번 조건 : 알파 보다 더 작은 베타가 존재해서 베타가 super key가 되면 안 됩니다. Super key 중에서 알파가 가장 작아야 합니다.
- **Candidate Key** 라면 **Super Key**입니다.
- **Primary Key** 라면 **Super Key**입니다.

<Example>

Example: Professor(pID, name, deptName, salary) we have

- pID → name deptName salary
 - pID → name
 - pID → deptName
 - pID → salary
 - pID → name deptName
 - ... and so on
- pID가 primary key이므로 pID가 같으면 속성들이 같아야 함을 보여줍니다.

<11-2강_Functional Dependency Theory> 12/12

- Functional Dependency에 대한 여러 개념을 배웁니다.

1. Trivial Functional Dependency (=당연한 Functional Dependency)

= 모든 instance에 대해서 Functional Dependency를 만족시키는 경우

- 베타의 속성이 알파에 포함되면 Trivial Functional Dependency입니다.

<Example>

In general, $\alpha \rightarrow \beta$ is trivial if $\beta \subseteq \alpha$

- Example

- ID name $\rightarrow$ ID
- name $\rightarrow$ name

① ID + name $\rightarrow$ ID

= ID와 name이 같으면 name이 같습니다.

- 너무 당연하므로 trivial Functional Dependency

② name $\rightarrow$ name

= name이 같으면 name이 같습니다.

- 너무 당연하므로 trivial Functional Dependency

2. Closure of a Set of Functional Dependency

= 주어진 Functional Dependency(F)를 통해 유도할 수 있는 새로운 Functional Dependency(F+)

- F에 비해 F+는 엄청 커지기 때문에 모든 원소를 찾는 것은 어려운 작업입니다.
- +를 closure라고 부릅니다. F+는 F closure.

1) F+ 구성요소

<F : {A->B , B->C} 라고 할 때 F+를 구해보시다.>

① F+에는 F도 포함됩니다.

- $F \subset F^+$

② F+에는 F를 보고 만든 새로운 Functional Dependency가 포함됩니다.

- A->B 이고 B->C 이므로 새로운 Functional Dependency인 A->C를 만들 수 있습니다.
- $\{A \rightarrow C\} \subset F^+$

③ F+에는 Trivial Functional Dependency가 포함됩니다.

- $\{A \rightarrow A, \{B \rightarrow B\} \subset F^+$

2) Armstrong's Axioms (암스트롱의 공리)

= 'F → F+'를 찾을 때 아래의 3가지 공리를 이용하면 모든 Functional Dependency를 찾을 수 있습니다.

- 공리 : 주어진 이론 체계 안에서는 증명 없이 True인 것으로 받아들이는 명제
- 항상 sound 하고 complete한 성질을 갖고 있습니다.
- Sound : 정상적인 Functional Dependency를 찾아냅니다.
- Complete : 긴 시간이 걸리지만 3가지 공리만을 이용하면 모든 Functional Dependency를 찾아낼 수 있습니다.

- if $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$ (reflexivity)
- if $\alpha \rightarrow \beta$, then $\gamma \alpha \rightarrow \gamma \beta$ (augmentation)
- if $\alpha \rightarrow \beta$ and $\beta \rightarrow \gamma$, then $\alpha \rightarrow \gamma$ (transitivity)

① Reflexivity : 베타가 알파에 포함되면 알파 → 베타가 성립합니다.

② Augmentation : 알파 → 베타가 성립하면 양변에 감마를 곱해도 상관없습니다.

③ Transitivity : 삼단논법

<Example : 주어진 정보에 3가지 공리를 사용하여 F+를 만들어 봅시다.>

R = (A, B, C, G, H, I)

F = {A → B

CG → H

A → C

CG → I

B → H}

- 속성 : A, B, C, G, H, I
- F : Functional Dependency

- A → H : A → B AND B → H ⇒ A → H
- AG → I : A → C ⇒ AG → CG ⇒ CG → I ⇒ AG → I
- CG → HI : CG → I ⇒ CGCG → CGI ⇒ CGCG → CG는 당연하므로 CG → CGI & CG → H ⇒ CG → HI
- A → BC도 가능합니다.

3) Additional Rules

= 3가지 공리만으로도 충분히 모든 Functional Dependency를 찾아 낼 수 있지만 좀 더 편하게 찾기 위해 만들어진 공식

Additional rules

- If $\alpha \rightarrow \beta$ and $\alpha \rightarrow \gamma$, then $\alpha \rightarrow \beta \gamma$ (union)
- If $\alpha \rightarrow \beta \gamma$, then $\alpha \rightarrow \beta$ and $\alpha \rightarrow \gamma$ (decomposition)
- If $\alpha \rightarrow \beta$ and $\gamma \beta \rightarrow \delta$, then $\alpha \gamma \rightarrow \delta$ (pseudotransitivity)

① Union : 알파 $\rightarrow$ 베타 AND 알파 $\rightarrow$ 감마 $\Rightarrow$ 알파 $\rightarrow$ 베타감마

② Decomposition : 알파 $\rightarrow$ 베타감마 $\Rightarrow$ 알파 $\rightarrow$ 베타 AND 알파 $\rightarrow$ 감마

③ Pseudo Transitivity : 알파 $\rightarrow$ 베타 AND 감마베타 $\rightarrow$ 델타 $\Rightarrow$ 알파 감마 $\rightarrow$ 델타

- 암스트롱의 공리로 3가지 추가 공식의 정당성을 모두 증명할 수 있습니다.

3. Closure of Attribute Sets

- '2. Closure of a Set of Functional Dependency'에서 배운 F^+ 찾기 방식은 시간이 너무 오래 걸립니다. 이번 챕터에서 배우는 건 빠릅니다.

<Example : $R = (A, B, C)$, $F = \{A \rightarrow B, B \rightarrow C\}$ >

① 이전의 방식

= Functional Dependency와 암스트롱의 공리를 이용하여 F^+ 를 마구잡이로 구합니다.

② 새로운 방식

= Attribute Sets($A^+, B^+, C^+, AB^+, BC^+, AC^+, ABC^+$)을 모두 구하고 합쳐서 F^+ 를 구합니다.

- A로 시작하는 Functional Dependency($A \rightarrow$ 속성)을 모두 구해서 A^+ 를 만들고, B로 시작하는 Functional Dependency($B \rightarrow$ 속성) B^+ 를 만들고, C로 시작, AB로 시작, BC로 시작, AC로 시작, ABC로 시작하는 Functional Dependency를 모두 구해서 합치면 F^+ 를 찾을 수 있습니다.

- A^+ 는 $A \rightarrow A, A \rightarrow B, A \rightarrow C$ 이므로 $A^+ = ABC$ 라고 표기할 수 있습니다.

- ' $A^+ = ABC$ '를 알파 = 베타로 표기하면, 베타는 A^+ 에서 $\rightarrow$ 뒤에 오는 모든 속성을 연속해서 압축하여 적습니다.

③ 새로운 방식의 알고리즘으로 A^+ 찾기

- 초기값 : $A \rightarrow A$
- 루프를 돌면서 $A \rightarrow$ 를 찾아야하는데 , 만약 $A \rightarrow B$ 존재하고 $B \rightarrow C$ 존재하면 $A \rightarrow C$ 이므로 A^+ 에 $A \rightarrow C$ 를 추가합니다.
- 포함시킬 때 마다 closure가 바뀌게 되므로 이전에 포함되지 못한 Functional Dependency가 포함될 수 있습니다.

<Example : AG^+ 를 구해보시다.>

$R = (A, B, C, G, H, I)$

- $F = \{A \rightarrow B \quad A \rightarrow C \quad CG \rightarrow H$
 $\quad \quad CG \rightarrow I \quad B \rightarrow H\}$

- ① $AG \subset AG^+$
- ② $AG \rightarrow AG$ 이므로 $AG \rightarrow A$ & $AG \rightarrow G$ 입니다.
- ③ $A \rightarrow B$ 이므로 $AG \rightarrow BG$ 이므로 $AG \rightarrow B$ & $AG \rightarrow G$ 입니다. 그러므로 B 포함 AG^+
- ④ 위와 동일한 과정을 계속하다보면 C , H , I도 AG^+ 에 포함시킬 수 있습니다.
- ⑤ $AG^+ = ABCGHI$ 이며 $AG^+ = R$ 이라고 할 수 있습니다.

- 이 문제에서 AG 가 Candidate Key가 되려면 $AG \rightarrow R$ 이지만 , AG 가 가장 최소의 속성이어야 합니다. AG 에 포함되며 AG 보다 작은 속성인 $A \rightarrow R$ OR $G \rightarrow R$ 이 성립하는지 검사해야 합니다. A^+ 와 G^+ 를 구해보면 R 이 아니므로 AG 는 Candidate Key입니다.

4. Canonical Cover

1) 정의

- $F = \{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$ 일 때 F 에서 중복된 Functional Dependency 존재하는지 탐색합니다.
- $A \rightarrow C$ 는 $A \rightarrow B$ & $B \rightarrow C$ 로 만들 수 있으므로 제거가 가능합니다.
- F 에서 중복되는 Functional Dependency를 점점 제거하여 더 이상 중복되는 Functional Dependency가 존재하지 않으면 F_c (F 의 Canonical Cover)라고 부릅니다.
- F 에서 중복된 정보를 지워서 F_c 를 만들었을 때 F 의 정보의 크기와 F_c 의 정보의 크기는 반드시 같아야합니다. 그러므로 F^+ 와 F_c^+ 가 같은지 검사해서 정보의 크기가 같은지 확인해야합니다. (아래의 2)_①에서 확인해봅시다.)

2) Canonical Cover 찾기

<Example_1 : $F = \{A \rightarrow B, B \rightarrow C, A \rightarrow CD\}$ >

- $\{A \rightarrow B, B \rightarrow C, A \rightarrow CD\}$ can be simplified to $\{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$

- $A \rightarrow CD$ 에서 중복이 발생합니다. C나 D를 제거 할 수 있는지 검사합니다.

① C 제거하기 : 제거 가능

- $A \rightarrow C$ 는 $A \rightarrow B$ & $B \rightarrow C$ 로 유도할 수 있으므로 C를 제거합니다.
- $F_c = \{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$ 이며, F^+ 와 F_c^+ 가 같은지 확인해봅시다.
- F_c 에서는 $A \rightarrow B$ & $B \rightarrow C$ 로 $A \rightarrow C$ 를 유도하고, $A \rightarrow CD$ 도 유도할 수 있습니다.
- F 에서는 $A \rightarrow B$ & $B \rightarrow C$ 로 $A \rightarrow C$ 를 유도하고, $A \rightarrow CD$ 이므로 $A \rightarrow C$ & $A \rightarrow D$ 를 유도할 수 있으므로 F^+ 와 F_c^+ 가 같습니다.
- C처럼 제거 가능한 속성을 extraneous attribute라고 합니다. 완벽하게 제거한 F_c 에는 extraneous attribute가 존재하지 않게 됩니다.

② D 제거하기 : 제거 불가능

- $A \rightarrow CD$ 에서 D를 제거하면 F_c 에서는 $A \rightarrow D$ 를 유도할 수 없으므로 F 와 F_c 의 정보의 차이가 생깁니다.

<Example_2>

$$\begin{array}{l} R = (A, B, C) \\ F = \{A \rightarrow BC, \\ \quad A \rightarrow B, \quad B \rightarrow C \\ \quad AB \rightarrow C\} \end{array}$$

① $A \rightarrow BC$, $A \rightarrow B$ 가 중복되므로 $A \rightarrow BC$ 만 남깁니다.

- $F_c = \{A \rightarrow BC, B \rightarrow C, AB \rightarrow C\}$

② $AB \rightarrow C$ 에서 A 를 제거하거나 , B 를 제거 할 수 있습니다. A 를 제거 합니다.

- A 를 제거하면 $F_c = \{A \rightarrow BC, B \rightarrow C(\text{중복})\}$ 가 됩니다.

- F_c 에서 $AB \rightarrow C$ 를 유도할 수 있는지 검사합니다.

- $B \rightarrow C \Rightarrow AB \rightarrow AC$ 이므로 $AB \rightarrow A$ & $AB \rightarrow C$ 를 유도할 수 있습니다.

- $F_c = \{A \rightarrow BC, B \rightarrow C\}$

③ $A \rightarrow BC$ 에서 B 를 제거하거나 , C 를 제거 할 수 있습니다. C 를 제거 합니다.

- C 를 제거하면 $F_c = \{A \rightarrow B, B \rightarrow C\}$ 가 됩니다.

- F_c 에서 $A \rightarrow BC$ 를 유도할 수 있는지 검사합니다.

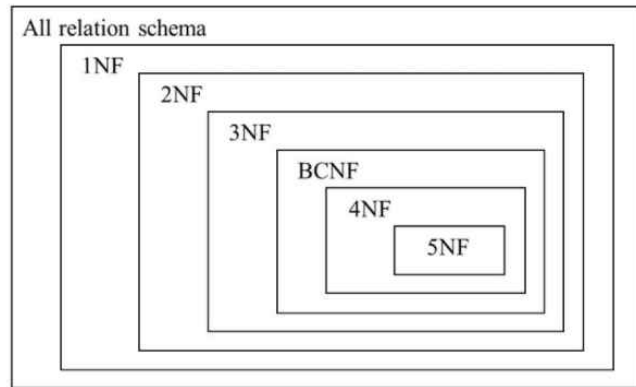
- $A \rightarrow B$ & $B \rightarrow C \Rightarrow A \rightarrow C$

- $A \rightarrow B$ & $A \rightarrow C \Rightarrow A \rightarrow BC$

- 최종 $F_c = \{A \rightarrow B, B \rightarrow C\}$

<11-3강_Normal Forms> 12/13

1. Overview



- 이번 강의는 정규형에 대해서 배웁니다.
- 정규형에는 총 6가지 종류가 존재합니다.
- '1NF → 2NF → 3NF → BCNF → 4NF → 5NF' 순서로 존재하며, 우측으로 갈수록 조건이 까다로워집니다.
- 강의에서는 BCNF까지 배웁니다.

2. First Normal Form (1NF)

= 제 1 정규형

= 테이블에 존재하는 모든 도메인이 atomic 하면 1NF

- Atomic 하지 않은 도메인 : sets , lists , bags , composite attributes
- 가장 약한 조건
- DB에 존재하는 모든 테이블이 1NF를 만족 시키면 1NF라고 합니다.

3. Various Functional Dependencies

- 2NF를 이해하기 위해 배워야 할 5가지 Functional Dependencies

1) Prime attribute

= 스키마_R에 대한 candidate key를 구성하는 속성

2) Non-prime attribute

= 스키마_R에 대한 candidate key를 구성하지 않는 속성

- Prime attribute의 여집합

3) Fully Functional Dependency (완전한 함수 종속)

= 알파 $\rightarrow$ 베타 AND 감마 $\subset$ 알파 $\Rightarrow$ 감마 $\rightarrow$ 베타가 존재하지 않은 경우

- 다시 말해, 알파 $\rightarrow$ 베타가 존재하는데 알파 보다 작은 속성이 베타와 Functional Dependency를 이루면 안 됩니다.

ex) $AB \rightarrow C$ 일 때 $A \rightarrow C$ 를 만족하지 않고, $B \rightarrow C$ 를 만족하지 않으면 Fully Functional Dependency

4) Partial Functional Dependency (부분적 함수 종속)

= 알파 $\rightarrow$ 베타 AND 감마 $\subset$ 알파 $\Rightarrow$ 감마 $\rightarrow$ 베타가 존재하는 경우

ex) $AB \rightarrow C$ 일 때 $A \rightarrow C$ 를 만족하지 않지만, $B \rightarrow C$ 를 만족하면 Partial Functional Dependency

5) Transitive Functional Dependency (이행적 함수 종속)

- 삼단논법 : 알파 $\rightarrow$ 베타 AND 베타 $\rightarrow$ 감마 $\Rightarrow$ 알파 $\rightarrow$ 감마

- 알파 $\rightarrow$ 감마를 Transitive Functional Dependency 관계라고 부릅니다.

4. Second Normal Form (2NF)

= 제 2 정규형

= 알파 $\rightarrow$ 베타 관계에서, 모든 non-prime attribute에 대해서 candidate key $\rightarrow$ non-prime attribute를 만족해야 합니다. 알파에 candidate key의 부분집합(자신 제외)이 왔을 때 만족하면 안 되고, 알파에 candidate key와 관련이 없는 속성이 왔을 때의 Functional Dependency는 무시

- 2NF는 당연히 1NF의 조건을 선행시켜야 합니다.

- Insertion, Update, Delete에서 중복이 발생할 수 있습니다.

<Example_1>

R(SSN, pNumber, hours, eName, pName, pLocation)

SSN pNumber $\rightarrow$ hours

SSN $\rightarrow$ eName

pNumber $\rightarrow$ pName pLocation

R1(SSN, pNumber, hours)

R2(SSN, eName)

R3(pNumber, pName, pLocation)

① 모든 도메인 atomic을 검사합니다.

② Candidate Key와 non-prime attribute를 찾습니다.

- Candidate Key : 밑줄 친 속성 $\rightarrow$ SSN , pNumber

- Non-prime attribute : 속성 중에서 Candidate Key 들을 제외한 속성 $\rightarrow$ hours , eName , pName , pLocation

- non-prime attribute의 4가지 멤버에 대해서 모든 Candidate Key에 대한 Fully Functional Dependency를 결정해야 합니다.

③ hours 검사

- 'SSN pNumber $\rightarrow$ hours'를 성립시키므로 2NF 조건을 만족시킵니다.

④ eName 검사

- 'SSN $\rightarrow$ eName'를 확인하면 SSN pNumber가 알파일 때 더 작은 감마인 SSN에 대해서 SSN $\rightarrow$ eName이 성립하므로 2NF 조건을 만족시키지 않습니다.

- 만족하지 않는 경우를 발견했으므로 스키마_R은 2NF를 만족시키지 못합니다.

⑤ pNumber $\rightarrow$ pName pLocation

- 만족 X

- 2NF를 만족 시키지 않는다면 분해(decompose)해야 합니다.

- 위의 그림은 2번의 분해가 일어납니다. 그 결과 R1 , R2 , R3은 2NF를 만족합니다.

<Example_2>

FIRST(S#, P#, Status, City, Qty)

$S\# \ P\# \rightarrow \text{Status} \ \text{City} \ \text{Qty}$

$S\# \rightarrow \text{City} \ \text{Status}$

$\text{City} \rightarrow \text{Status}$

- 스키마_FIRST
- Candidate key : S# , P#
- Non-prime attribute : Status , City , Qty

① 1번 Functional Dependency : 2NF를 만족시킵니다.

② 2번 Functional Dependency : 2NF를 만족시키지 않습니다.

= $S\# \rightarrow \text{City}$ & $S\# \rightarrow \text{Status}$ 로 나눌 수 있으며 두 가지 Functional Dependency는 2NF를 만족시키지 않습니다.

③ 3번 Functional Dependency : 2NF와 관련이 없습니다.

④ 2NF를 위해 decompose를 해봅시다.

FIRST is not in 2NF, so decompose into

SECOND(S#, Status, City)

SP(S#, P#, Qty)

$S\# \rightarrow \text{City} \ \text{Status}$

$\text{City} \rightarrow \text{Status}$

$S\# \ P\# \rightarrow \text{Qty}$

5. Third Normal Form (3NF)

= 제 3 정규형

= 조건_1 : 어떠한 non-prime attribute도 candidate key에 대해서 Transitive Functional Dependency를 만족시키지 않으면 3NF.

= 조건_2 : 모든 Functional Dependency에 대해서 알파 -> 베타에서 알파가 super key OR 베타가 prime attribute를 만족시키면 3NF

- 3NF를 판별하는 두 가지 조건은 완전히 동일하므로 , 둘 중 하나만 검사해서 만족시켜도 그 스키마는 3NF가 됩니다.

- 3NF는 당연히 2NF의 조건을 선행시켜야 합니다.

<Example_1>

myEmpDept(SSN, Ename, Bdate, Addr, D#, Dname, Dmgr)

SSN → Ename Bdate Addr D#

D# → Dname Dmgr

- Candidate Key : SSN

- Non-prime key : Ename , Bdate , Addr , D# , Dname , Dmgr

1) 조건_1로 검사

① Ename , Bdate , Addr , D#

- 'SSN -> Ename' → 'Ename -> 속성' 이 존재하지 않습니다.

- Bdate , Addr도 마찬가지입니다.

② Dname

- 'SSN -> D# -> Dname'이 성립하여 'SSN -> Dname'의 Transitive Functional Dependency가 생성됩니다.

- 그러므로 스키마_myEmpDept는 3NF를 만족시키지 않습니다.

2) 조건_2로 검사

① SSN → Ename Bdate Addr D#

- 알파가 candidate key 이므로 super key 이므로 OR 조건에 따라 베타를 검사하지 않고도 조건_2를 만족시킵니다.

② D# → Dname Dmgr

- 알파가 super key가 아닙니다. 베타는 prime attribute key도 아닙니다. 그러므로 조건_2를 만족시키지 않습니다.

3) 분해해서 3NF 만족시키기

ED1(SSN, Ename, Bdate, Addr, D#)
ED2(D#, Dname, Dmgr)

<Example_2>

SECOND(S#, Status, City)

SP(S#, P#, Qty)

S# → City Status

City → Status

- SECOND와 SP는 2NF의 예제이므로 2NF는 만족시킵니다.

① SECOND

= S# → City → Status로 transitive functional dependency를 만족시키므로 3NF가 아닙니다.

② SP

= 3NF를 만족시킵니다.

6. Boyce/Codd Normal Form (BCNF)

= BC 정규형

= 3NF의 두 번째 조건 중에서 앞의 조건만 만족하면 됩니다.

- 다시 말해 , 주어진 모든 Functional Dependency에 대해서 알파 -> 베타에서 알파가 Super Key라면 BCNF를 만족합니다.
- BCNF는 제 3정규형보다 더 강한 조건입니다.
- 모든 Binary Relations는 BCNF를 만족시킵니다.

<Example>

MovieStudio(title, year, length, filmType, studioName, studioAddr)

title year → length filmType studioName

studioName → studioAddr

Candidate key: (title, year)

Hence, MovieStudio is not 3NF

- 2NF를 만족 시킵니다.

① 3NF

- 3NF는 만족시키지 않습니다.
- 'Title year -> studioName -> studioAddr' 이므로 Transitive Functional Dependency가 존재합니다.

② BCNF

- 직접 테스트 해봅시다.

*** 교재 연습문제 풀어보고 모르는 문제 Database Design Theory (8) 듣고 공부하기 ***

<11-4강_Decomposition and Design> 12/13

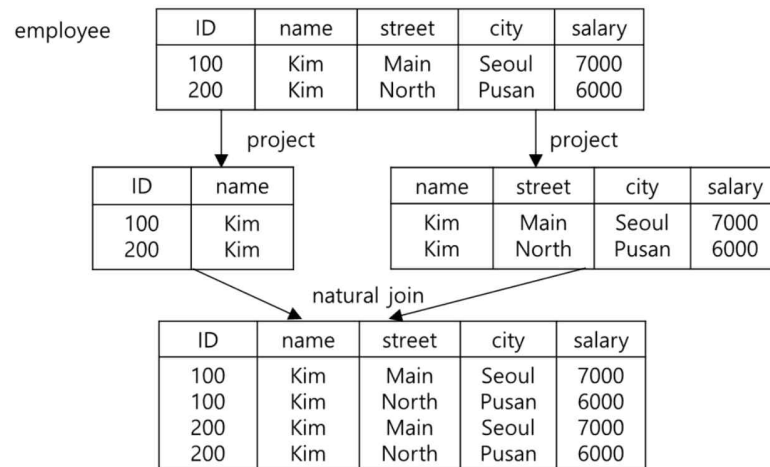
- 정규화를 만족시키지 않을 때마다 우리는 decomposition(분해)해서 만족합니다.
- 그러나 마구잡이로 분해를 하면 안 되고 반드시 지켜야 하는 조건이 있습니다.

1. 분해의 목적

- 사람마다 Good 스키마를 정의하는 것이 다르므로, 해당 Good 스키마가 요구하는 레벨의 정규화를 만족시키기 위해 분해를 해야 합니다.
- 분해를 하는 과정에서 반드시 **lossless-join**을 하여야 합니다.
- 분해를 하는 과정에서 **dependency preserving**을 하면 좋을 것 같습니다. (반드시 만족할 필요는 없습니다.)

2. Lossy Decomposition

= 올바르게 못한 분해



- R(employee table)을 두 개의 table_R1, table_R2 분해합니다.
- R1과 R2를 natural join하여 table_R3를 만들었습니다.
- R과 R3의 정보의 양이 육안으로 보아도 다릅니다.
- 그러므로 R1과 R2는 잘못 분해된 Table입니다.

3. Lossless-Join Decomposition

= R을 R1 , R2로 분해했을 때 'R1 ∩ R2 의 원소가 R1에서 super key OR R1 ∩ R2 의 원소가 R2에서 super key'를 만족하면 Lossless-Join Decomposition을 만족하고 , 이는 올바른 분해가 됩니다.

- 모든 테이블에 대해서 분해하고 join 하여 원본과 비슷한지를 검사하는 것은 매우 복잡합니다. 그러므로 Lossless-Join Decomposition을 적극 활용하여 쉽게 검사합니다.

- Lossless-Join Decomposition 덕분에 분해하고 Join하는 연산을 할 필요가 없으며 , 이는 올바르게 분해하였는지 검사하는 좋은 방법입니다.

4. Dependency Preservation

- R을 R1 , R2 , etc , Rn으로 분해합니다.

- F : R의 속성들이 만족시켜야 하는 Functional Dependency

- F1 : R1의 속성들이 만족시켜야 하는 Functional Dependency

- F2 ~ Fn을 F1과 동일한 방식으로 만듭니다.

$$(F_1 \cup F_2 \cup \dots \cup F_n)^+ = F^+$$

- F1 ~ Fn까지의 closure는 F의 closure와 동일해야 합니다.

- Dependency Preservation은 Lossless-Join과는 다르게 필수가 아닙니다. 하지만 , Dependency Preservation을 만족하지 않는 스키마는 Functional Dependency를 검사하는 과정에서 Join을 하는 경우가 발생할 수 있습니다.

*** Super Key를 찾을 때 Candidate Key를 확인하고 , Super Key의 정의도 확인해야 합니다. 그러므로 총 2가지 확인 ***

5. 분해 예시

$$R = (A, B, C)$$

$$F = \{A \rightarrow B, B \rightarrow C\}$$

1) 분해 경우_1

$$R_1 = (\underline{A}, B), \quad R_2 = (\underline{B}, C)$$

① Lossless-Join Decomposition : O

- $R_1 \cap R_2$ 는 속성_B입니다.
- B가 R_1 에서는 candidate key가 아니므로 super key가 아니지만, R_2 에서는 candidate key 이므로 super key입니다.

② Dependency Preserving : O

= $A \rightarrow B$ 는 R_1 에서 테스트 할 수 있고, $B \rightarrow C$ 는 R_2 에서 테스트 할 수 있습니다.

2) 분해 경우_2

$$R_1 = (\underline{A}, B), \quad R_2 = (\underline{A}, C)$$

① Lossless-Join Decomposition : O

- $R_1 \cap R_2$ 는 속성_A입니다.
- A가 R_1 에서는 candidate key 이므로 super key입니다. OR 조건이므로 R_2 에서 검사할 필요가 없습니다.

② Dependency Preserving : X

- $A \rightarrow B$ 를 R_1 에서 테스트 할 수 있습니다. 하지만 $B \rightarrow C$ 는 테스트를 하려면 R_1 과 R_2 를 join 해야 합니다. 테스트를 위해 Join이 발생하면 Dependency Preserving를 만족하지 않습니다.
- Test를 하는 경우 가급적 Join이 없도록 해야 합니다.

6. 3NF & BCNF

1) 3NF

- 3NF는 언제나 Lossless-Join Decomposition을 보장합니다.
- 3NF는 언제나 Dependency Preserving을 보장합니다.
- 하지만 , 중복이 발생하여 3가지 Anomaly(insert , update , delete)가 발생할 수 있습니다.

2)

- BCNF는 언제나 Lossless-Join Decomposition을 보장합니다.
- BCNF는 언제나 Dependency Preserving을 보장하지는 않습니다.
- 하지만 , 중복이 발생하지 않기 때문에 3가지 Anomaly(insert , update , delete)가 발생하지 않습니다.